

Bordeaux INP ENSEIRB-MATMECA
Formation d'ingénieur Systèmes Électroniques Embarqués

Mémoire de fin d'études

présenté par

SAUTEREAU Valentin

dans le cadre de la formation d'ingénieur de l'ENSEIRB-MATMECA dans la
spécialité Systèmes Électroniques Embarqués

Optimisation et extension fonctionnelle du banc de simulation pour drone

soutenu le **29 juin 2026**

Confidentialité	Entreprise
Rapport : ☐ oui ☒ non	THALES AVIONICS
Soutenance : ☐ oui ☒ non	75 - 77 avenue Marcel Dassault
	33700 Mérignac
	05 24 44 64 00

Maître d'Apprentissage
FEIJOEIRO Alexandre
alexandre.feijoeiro@thalesgroup.com

Tuteur Pédagogique
GIRAUD Rémi
remi.giraud@enseirb-matmeca.fr

Optimisation et extension fonctionnelle du banc de simulation pour drone

Sujet du mémoire : Ma mission, au travers de ce mémoire, a consisté à améliorer un banc de simulation drone. J'ai pour cela intégré le simulateur tactique VBS4, développé une tourelle optronique motorisée et mis en place des outils d'exploitation vidéo. L'objectif stratégique est double. Il s'agit d'une part de pouvoir valider des fonctions optroniques complexes au sein d'un environnement simulé hautement représentatif. D'autre part, l'enjeu est de capitaliser sur ces développements en concevant un socle modulaire, dont les différentes briques technologiques pourront être réutilisées sur d'autres bancs de simulation.

Résumé : Ce projet a pour but de faire évoluer notre banc de simulation afin de reproduire fidèlement le comportement d'un drone et de sa charge optronique. En effet, l'intégration de VBS4 me permet désormais d'injecter des données de vol en temps réel tout en générant un flux vidéo simulé. Pour aller plus loin, j'ai conçu un support motorisé qui reproduit physiquement les mouvements du drone. Cette action permet de stimuler directement la centrale inertielle de la tourelle, rendant enfin possible la validation des fonctions de suivi de cible en simulation.

Pour répondre au besoin d'aide à la décision, j'ai développé un module calculant et affichant la fauchée caméra au sol en temps réel. Le flux vidéo généré est ensuite exploité pour éprouver les algorithmes de détection IA (véhicules, cibles). Ce dispositif global nous permet de valider les performances des systèmes embarqués avant tout essai en vol.

PROJET

Précédente version	Nouvelle version
– Simulation en environnement non tactique – Interface avec données de simulation drone disponible – Pas de support de tourelle motorisé – Pas de stimulation de la centrale inertielle – Path tracking et geo tracking non validables sur banc – Pas de visualisation de la fauchée caméra – Flux vidéo simulé non exploité	– Intégration du simulateur tactique VBS4 – Injection des données temps réel – Simulation de scénarios opérationnels – Support de tourelle motorisé (Hardware-in-the-Loop) – Stimulation de la centrale inertielle – Validation des fonctions optroniques – Path tracking – Geo tracking – Calcul et affichage de la fauchée caméra – Exploitation du flux vidéo simulé – Détection IA de véhicules et cibles

Mots-clés : Hardware-in-the-Loop (HIL) / VBS4 / Plugin C++ / UDP multicast / Télémétrie temps réel / MPEG-TS / MAVLink / Microcontrôleur / Moteur pas-à-pas / Encodeur magnétique AS5600 / Bus I2C / Contrôle en boucle fermée / Traitement d'image / Simulation temps réel / Systèmes embarqués / Optronique

Optimisation et extension fonctionnelle du banc de simulation pour drone

ANNÉES

2023-2026

AUTEUR

Valentin Sautereau

MAÎTRE D'APPRENTISSAGE

Alexandre Feijoeiro

IVVQ Manager

ENTREPRISE

Thales AVIONICS

75 - 77 avenue Marcel Dassault, 33700 Mérignac, France

TUTEUR PÉDAGOGIQUE

Rémi Giraud

RÉSUMÉ

Mon projet a consisté à transformer un banc de simulation statique en un véritable démonstrateur *Hardware-in-the-Loop* (HIL) dédié à la validation des charges optroniques embarquées. En intégrant le simulateur tactique VBS4, j'ai mis en place une architecture capable d'injecter des données de vol en temps réel tout en générant un flux vidéo multispectral. Afin de lever le verrou de l'immobilité matérielle, j'ai conçu un support motorisé asservi en boucle fermée qui reproduit physiquement les mouvements du drone. Cette action permet de stimuler directement la centrale inertielle de la tourelle, rendant ainsi possible la validation de fonctions avancées telles que le *path tracking* et le *geo tracking* en simulation. Par ailleurs, le banc intègre un module de calcul de fauchée caméra et sert de plateforme de test pour les algorithmes de détection IA. Ce dispositif global constitue désormais un socle technologique modulaire, conçu pour capitaliser sur ces briques logicielles et matérielles afin de les réutiliser sur les futurs bancs de simulation du groupe.

MOTS-CLÉS

Hardware-in-the-Loop (HIL) / VBS4 / Plugin C++ / UDP multicast / Télémétrie temps réel / MPEG-TS / MAVLink / Microcontrôleur / Moteur pas-à-pas / Encodeur magnétique AS5600 / Bus I2C / Contrôle en boucle fermée / Traitement d'image / Simulation temps réel / Systèmes embarqués / Optronique

Optimization and functional extension of the drone simulation test bench

YEARS

2023-2026

AUTHOR

Valentin Sautereau

APPRENTICESHIP SUPERVISOR

Alexandre Feijoeiro

IVVQ Manager

COMPANY

Thales AVIONICS

75-77 avenue Marcel Dassault, 33700 Mérignac, France

ACADEMIC TUTOR

Rémi Giraud

ABSTRACT

My project consisted in transforming a static simulation bench into a true *Hardware-in-the-Loop* (HIL) demonstrator dedicated to the validation of embedded optronic payloads. By integrating the VBS4 tactical simulator, I implemented an architecture capable of injecting real-time flight data while generating a multi-spectral video stream. To overcome the limitations of hardware immobility, I designed a closed-loop motorized support that physically reproduces the drone's movements. This action directly stimulates the turret's internal inertial measurement unit (IMU), thereby enabling the validation of advanced functions such as *path tracking* and *geo-tracking* within a simulation environment. Furthermore, the bench integrates a camera footprint calculation module and serves as a testing platform for AI detection algorithms. This global system now constitutes a modular technological foundation, designed to capitalize on these software and hardware building blocks for reuse in the Group's future simulation benches.

KEYWORDS

Hardware-in-the-Loop (HIL) / VBS4 / C++ plugin / UDP multicast / Real-time telemetry / MPEG-TS / MAVLink / Microcontroller / Stepper motor / AS5600 magnetic encoder / I2C bus / Closed-loop control / Image processing / Real-time simulation / Embedded systems / Optronics

Remerciements

Je tiens tout d'abord à remercier sincèrement mon maître d'apprentissage, Alexandre Feijoeiro, ainsi qu'Eric Valade, mon précédent maître d'apprentissage. Leur encadrement, la confiance qu'ils m'ont accordée et les responsabilités qu'ils m'ont confiées ont été déterminants. Leur accompagnement m'a permis de m'intégrer pleinement à l'équipe Drone et de piloter ce projet de développement sur banc avec une réelle autonomie, dans un environnement aussi exigeant que formateur.

Je remercie également Thales Avionics pour son accueil. Un grand merci à l'ensemble de l'équipe Drone pour leur disponibilité, leurs conseils avisés et la richesse de nos échanges techniques.

J'adresse ma reconnaissance à mon tuteur pédagogique, Rémi Giraud, pour son suivi attentif, ses recommandations et son accompagnement constant.

Je remercie bien évidemment les équipes pédagogiques de l'ENSEIRB-MATMECA. La qualité des enseignements dispensés m'a permis d'acquérir le bagage technique nécessaire pour mener à bien ce travail de conception.

Je tiens aussi à remercier chaleureusement Andrew Taberner, directeur du laboratoire de bioingénierie d'Auckland (Nouvelle-Zélande), et ses équipes. L'accueil qu'ils m'ont réservé durant mes trois mois de mobilité internationale a rendu cette expérience particulièrement enrichissante sur le plan humain et scientifique.

Enfin, j'exprime ma gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet et à mon développement professionnel.

Table des matières

1	Introduction	12
2	Contexte industriel et environnement du projet	13
2.1	Thales, leader de l'électronique	13
2.1.1	Le groupe Thales	13
2.1.2	Historique du groupe	14
2.1.3	La division aéronautique	14
2.1.4	Le campus Thales Mérignac	16
2.1.5	Le département OSA	17
2.1.6	L'équipe intégration charge utile	17
2.2	Contexte technique	19
2.2.1	Présentation des drones et charges optroniques	19
2.2.2	Fonctionnement d'une charge optronique	19
2.2.3	Fonctions optroniques : path tracking et geo tracking	20
2.2.4	Contraintes de validation	20
2.2.5	Limites des essais en conditions réelles	21
2.2.6	Intérêt des bancs de simulation	21
3	Présentation du projet et analyse de l'existant	22
3.1	Le banc de simulation comme levier d'innovation (PoC)	22
3.1.1	Objectifs du banc de simulation	23
3.1.2	Architecture existante	23
3.1.3	Interface avec le simulateur et limites de la visualisation	24
3.1.4	Flux de données et Data Distribution	25
3.1.5	Fonctionnement global du banc statique	26
3.2	Analyse des limitations et nécessité d'une rupture	26
3.2.1	Limites de l'immobilité matérielle : le verrou de l'IMU	26
3.2.2	Limites tactiques et géométriques du simulateur civil existant	27
3.2.3	La chaîne d'exploitation vidéo et l'intégration Magellium	27
3.2.4	Conséquences opérationnelles : la dépendance aux essais en vol	28
3.3	Objectifs et spécifications du projet	28
3.3.1	Objectifs fonctionnels et opérationnels	28
3.3.2	Spécifications techniques et modularité	29
3.3.3	Contraintes système et performance	29
4	Gestion de projet et méthodologie de travail	30
4.1	Approche méthodologique : le Cycle en V itératif	30
4.2	Organisation du travail et outils	30
4.2.1	Outils de conception et environnement technique	31
4.2.2	Coordination et dépendances extérieures	31
4.2.3	Planification	31
4.3	Analyse et gestion des risques	32
4.4	Communication et partage de connaissances	32
5	Conception et développement du banc de simulation Hardware-in-the-Loop	33
5.1	Architecture globale du système	33
5.1.1	Principe du Hardware-in-the-Loop (HIL)	33
5.1.2	Architecture matérielle	33

5.1.3	Architecture logicielle	34
5.1.4	Flux de données	34
5.2	Environnement Virtuel : Intégration du simulateur tactique VBS4	34
5.2.1	Justification du choix et rupture avec la simulation civile	34
5.2.2	Architecture d'ingestion et écoute réseau Multicast	35
5.2.3	Développement du plugin C++ : Architecture et gestion des flux	36
5.2.4	Résolution du verrou technique : Calcul dynamique du Cap (Yaw)	37
5.2.5	Interfaçage et synchronisation de la charge utile	38
5.2.6	Synthèse de l'intégration et perspectives multi-flux STANAG 4609	39
5.3	Environnement Physique : Support motorisé et système de contrôle	40
5.3.1	Conception mécanique et modélisation sous Fusion 360	40
5.3.2	Architecture électrique et gestion de la puissance moteur	41
5.3.3	Développement du firmware Teensy 4.1 et gestion de la latence	42
5.3.4	L'évolution vers la boucle fermée : Bus I2C et Encodeurs AS5600	43
5.3.5	Transition vers l'asservissement : La régulation PID	44
5.3.6	Synthèse des résultats et apport opérationnel	45
5.4	Intégration de la chaîne de traitement vidéo et de l'IA (Magellium)	45
5.4.1	Cadre de la collaboration et architecture réseau	46
5.4.2	Investigation technique : diagnostic des bugs de saturation	46
5.5	Calcul et affichage de la fauchée caméra (Footprint)	46
5.5.1	Modélisation géométrique et aide à la décision	47
5.5.2	Le choix du langage : de la flexibilité du Python à la performance du C	47
5.5.3	Algorithme de projection trigonométrique et matrices de rotation	48
5.5.4	Implémentation logicielle et interface graphique (SDL2)	48
5.5.5	Résultats et impact opérationnel	49
6	Validation et résultats	50
6.1	Stratégie de validation et méthodologie IVV	50
6.1.1	Objectifs et critères de réussite	50
6.1.2	Approche de test incrémentale	50
6.2	Résultats expérimentaux et analyse des performances	51
6.2.1	Validation de l'environnement virtuel et du plugin C++	51
6.2.2	Performance du support motorisé et asservissement	51
6.2.3	Qualification de la brique de détection IA (Magellium)	51
6.2.4	Impact de l'affichage de fauchée caméra	52
6.3	Analyse des difficultés techniques et d'intégration	53
6.3.1	Développement bas niveau et support API	53
6.3.2	Instabilité mécanique et inertie de la charge	53
6.3.3	Investigation des anomalies de saturation mémoire	53
6.4	Solutions pérennes et retour d'expérience	53
7	Conclusion et perspectives	54
7.1	Synthèse des travaux réalisés	54
7.2	Perspectives d'amélioration et évolutivité	54
7.3	Applications futures et transversalité	54
7.4	Retour d'expérience : une transition vers l'ingénierie système	55
8	Glossaire	56
9	Bibliographie	59

A	Annexes	59
A.1	Architecture détaillée	59
A.2	Architecture réseau et plan d'adressage	61
A.3	Protocoles et structures de trames	61
A.4	Schéma de câblage électronique	63
A.5	Extraits de code critiques	63

Table des figures

1	Implantation de Thales dans le monde	13
2	Historique de Thales	14
3	MRTT A330 de la France en train de ravitailler	15
4	Campus Thales Mérignac	16
5	Organigramme du service Thales AVS orienté métier	17
6	Organigramme du service Thales AVS orienté projet	18
7	Banc de Simulation drone existant	24
8	Tourelle Optronique immobile sur Banc	27
9	Diagramme de Gantt simplifié montrant les phases du projet	32
10	Mise en place d'un scénario tactique complexe avec unités mobiles dans VBS4 (planification dans l'éditeur et déroulement temporel de l'action). . .	35
11	Architecture fonctionnelle du plugin <code>Simu_Drone.cpp</code> et flux de données inter-systèmes.	36
12	Architecture du plugin : Interception, mise en buffer et lissage du flux de télémétrie.	37
13	Déduction géométrique du cap (Yaw) par comparaison de deux trames GPS successives.	38
14	Interface de configuration des offsets caméra et rendu multi-spectral (Visible/IR) synchronisé dans le simulateur VBS4.	39
15	Capture du flux vidéo VBS4 exporté en temps réel et lu via un lecteur réseau (UDP ://@224.2.0.1 :8485).	40
16	Conception et réalisation du support motorisé bi-axes	41
17	Driver de puissance TB6600 utilisé pour le pilotage des moteurs NEMA 17.	42
18	Architecture du flux de données : de la réception MAVLink UDP à l'asservissement en boucle fermée via encodeurs AS5600.	43
19	Encodeur magnétique rotatif AS5600 destiné à la mesure de position réelle des axes.	44
20	Modélisation de la correction d'erreur par PID : passage d'un système divergent (sauts de pas) à un suivi de consigne stabilisé.	45
21	Interface de l'application de détection automatique développée par Magellium.	46
22	Comparatif illustratif des ordres de grandeur du temps de cycle d'exécution.	47
23	Modélisation de la projection : la pyramide de vision issue du drone intersecte le plan terrestre pour former l'emprise au sol.	48
24	Capture d'écran du simulateur VBS4 illustrant le polygone de la fauchée projeté sur le relief (en rouge) lors d'une navigation en vue à la troisième personne.	49
25	Représentation du processus de validation incrémentale appliqué au banc HIL.	51
26	Analyse de robustesse sur 4 heures : la mise en place d'une purge cyclique (courbe orange) maintient la stabilité de l'occupation mémoire et de la charge CPU (courbe bleue).	52
27	Interface de détection IA validée (Magellium) : identification, classification (<i>car</i>) et suivi dynamique d'un convoi de véhicules au sein de l'environnement tactique VBS4.	52
28	Architecture système détaillée du banc de simulation HIL et cartographie complète des flux de données.	60

29	Structure standardisée du message MAVLink ATTITUDE (ID 30) contenant les angles de Roulis, Tangage et Lacet en radians.	61
30	Structure de l'encapsulation des métadonnées KLV (STANAG 4609) dans un flux de transport MPEG-TS.	62
31	Analyse du port série illustrant la conversion et l'envoi des trames de commande en centièmes de degrés.	62
32	Schéma de câblage intégrant la Teensy 4.1, les drivers TB6600 et le bus I2C des encodeurs AS5600.	63

Liste des tableaux

1	Clients et concurrents de Thales AVS	16
2	Matrice des risques et plans d'actions associés.	32
3	Comparaison des performances mécaniques : Impact de la réduction sur le couple et la précision.	42
4	Plan d'adressage réseau du banc de simulation.	61

1 Introduction

Aujourd'hui, les drones occupent une place véritablement stratégique dans de nombreux secteurs, allant de la défense à l'observation civile. Ces plateformes embarquent des charges optroniques complexes, conçues pour l'acquisition vidéo et le suivi de cibles. Pour garantir la fiabilité de ces équipements de pointe, nous devons nous appuyer sur des moyens de test rigoureux. En effet, il est indispensable de pouvoir reproduire fidèlement les conditions d'utilisation réelles. Dans ce contexte, Thales s'impose comme un acteur majeur des hautes technologies. La division Thales Avionics, et plus particulièrement le département *Optronics and Surveillance Applications* (OSA), développe et intègre des équipements destinés aux drones.

Cependant, valider ces systèmes en conditions réelles présente des contraintes lourdes. Coûts, logistique complexe, difficulté de reproductibilité : les obstacles sont nombreux. Pour répondre à ce besoin d'efficacité, les bancs de simulation constituent une alternative redoutable. Ils nous permettent de modéliser le comportement du drone et de sa charge utile dans un cadre maîtrisé. C'est sur ces bancs que nous pouvons tester les algorithmes d'image et les fonctions optroniques (telles que le *path tracking* ou le *geo tracking*) bien avant tout vol réel.

Mon mémoire s'inscrit précisément dans cette dynamique de modernisation. J'ai eu pour mission d'améliorer un banc de simulation existant afin d'y intégrer une architecture *Hardware-in-the-Loop* (HIL). L'objectif principal a été double : intégrer le simulateur tactique VBS4 pour générer des données visuelles crédibles, et concevoir un support motorisé pour reproduire physiquement les mouvements de l'aéronef. Cette action mécanique permet de stimuler directement la centrale inertielle de la charge optronique. Ainsi, nous pouvons valider son comportement dans des conditions d'utilisation réalistes. À cela s'ajoute l'exploitation du flux vidéo simulé et la création d'un module de calcul de la fauchée caméra, projetant l'empreinte au sol de la caméra en temps réel.

J'ai mené ce projet au sein de l'équipe intégration charge utile du département OSA à Mérignac. Ma responsabilité a consisté à concevoir, développer et interfacer l'ensemble des composants matériels et logiciels nécessaires à cette rupture technologique. De l'intégration de VBS4 au développement du support motorisé contrôlé par microcontrôleur en boucle fermée, j'ai agi en tant qu'architecte système. Ce travail a nécessité de croiser des compétences en électronique, en systèmes embarqués et en intégration logicielle.

Dans ce document, je présenterai d'abord le contexte industriel et l'environnement de mon projet. J'analyserai ensuite le banc historique pour en exposer les limites, ce qui justifiera les objectifs fixés. Par la suite, je détaillerai la conception du banc HIL : de l'architecture système à l'intégration de VBS4, en passant par le support motorisé et l'exploitation vidéo. Enfin, j'exposerai les résultats de validation obtenus, avant de conclure sur les perspectives qu'offre ce nouveau démonstrateur.

2 Contexte industriel et environnement du projet

2.1 Thales, leader de l'électronique

2.1.1 Le groupe Thales

De son ancien nom Thomson-CSF, Thales est un groupe d'électronique français présent sur 68 pays (voir figure 1), avec plus 83 000 collaborateurs dans le monde.



FIGURE 1 – Implantation de Thales dans le monde

Le groupe Thales est spécialisé dans plusieurs domaines :

- **Aéronautique** : fournir des systèmes et des équipements avioniques, gestion du trafic aérien, systèmes de formations et de simulations
- **Espace** : concevoir, développer et déployer des infrastructures orbitales, des systèmes satellites, des segments sol et des services associés pour les programmes spatiaux
- **Transport terrestre** : optimiser la capacité et l'efficacité des réseaux de transport terrestre dans des conditions de sécurité optimales, à un coût moindre et en offrant de meilleurs services aux voyageurs
- **Défense** : aider les forces armées à acquérir et conserver la supériorité décisionnelle et opérationnelle sur les théâtres conventionnels, le combat urbain et le cyberspace
- **Sécurité** : fournir des solutions intégrées, des réseaux fiables et des services à haute valeur ajoutée permettant de protéger les citoyens, les données sensibles et les infrastructures critiques

Depuis quelques années, Thales investit massivement dans 4 technologies clés pour accélérer la transformation digitale de ses clients : IA - Connectivité - Big Data - Cybersécurité. Thales est un acteur majeur dans la recherche et développement. Elle comptabilise à ce jour plus de 20500 brevets. Fort de sa R&D (dont 1,1 milliard d'euros autofinancé), le Groupe Thales aide ses clients à maîtriser des environnements complexes pour prendre des décisions rapides, efficaces, à chaque moment décisif.

2.1.2 Historique du groupe

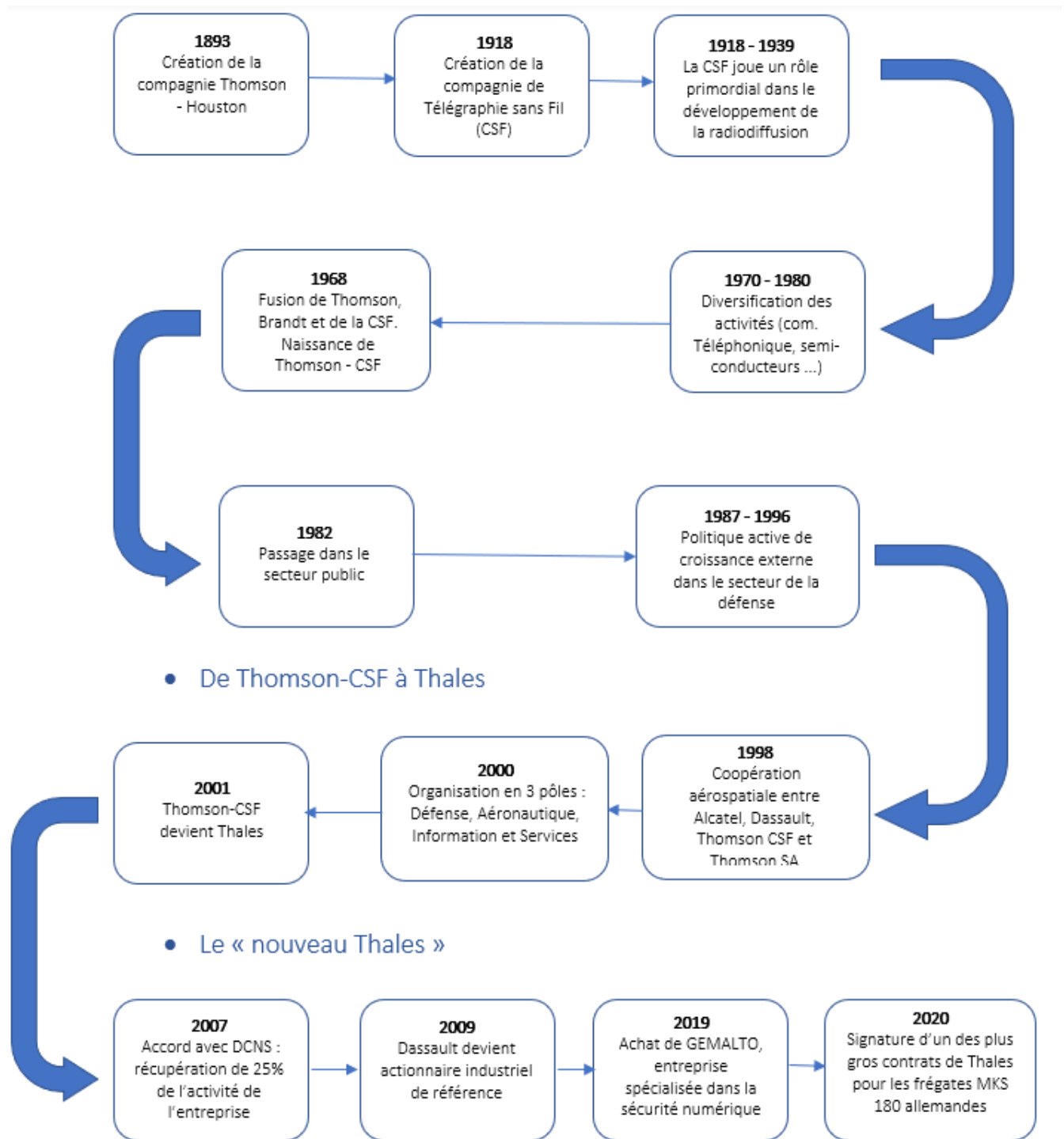


FIGURE 2 – Historique de Thales

2.1.3 La division aéronautique

Thales est présent dans 5 secteurs d'activités à savoir l'aéronautique, le spatial, le transport terrestre, la défense et la sécurité. L'entité à laquelle je suis rattaché est **Thales Avionics**, elle représente la division aéronautique civile.

Thales AVS est un acteur majeur dans les domaines de l'aéronautique civile. Ses activités sur ces marchés regroupent des systèmes avioniques, des systèmes de gestion du trafic aérien et des radars de surveillance, du multimédia de bord et des suites avionique

de cockpit, mais aussi des systèmes de simulation et de formation. Ses principales activités sont citées ci-dessous :

- **Gestion du trafic aérien** : Thales est le leader mondial pour les solutions de gestion de trafic. Aujourd'hui, 2 avions sur 3 décollent et atterrissent grâce à un équipement Thales.
- **Avionique de vol** : Thales est l'un des plus grands fournisseurs mondiaux d'électronique embarquée à bord des plateformes des principaux avionneurs : suite avionique, visualisation, instruments et fonctions de pilotage.
- **Multimédia et connectivité** : Les solutions de Thales équipent tous les types d'avions monocouloirs et bi-couloirs Airbus et Boeing. Elles ont été choisies par de nombreuses compagnies aériennes internationales. Thales est un des principaux fournisseurs mondiaux de solutions connectées.
- **Systèmes électriques** : Les technologies développées par Thales permettent d'améliorer, d'amplifier, d'automatiser et de contrôler les systèmes électriques à bord d'un aéronef. Thales fait partie des 3 premiers industriels mondiaux dans ce domaine.
- **Solution de navigation** : Thales possède l'expertise sur l'architecture système à bord d'un aéronef qui comprend l'inertie, les données aérométriques, les systèmes de positionnement GPS/GNSS qui permettent d'augmenter les performances à court, moyen et long terme afin de réduire l'impact écologique de l'avion.



FIGURE 3 – MRTT A330 de la France en train de ravitailler

Sur ces marchés, Thales AVS travaille avec plusieurs entreprises d'envergure internationale, des institutions gouvernementales ou des villes à travers le monde. Les principaux clients et concurrents sont listés ci-dessous (voir tableau 1).

Principaux clients	Principaux concurrents
Entreprises : • Airbus • Boeing • Comac • Dassault • Bombardier Institutions, gouvernements	Rockwell Collins Honeywell Garmin Northrop Grumman Raytheon Leonardo

TABLE 1 – Clients et concurrents de Thales AVS

2.1.4 Le campus Thales Mérignac

Regroupement de deux anciens sites (Pessac et Haillan), le site de Mérignac (voir figure 4) a ouvert ses portes en 2016. Il comptabilise environ 2500 employés tout contrats confondus et figure parmi les premiers employeurs de la région. Deux entités sont présentes sur ce site :

- Thales AVS : Systèmes civils / Avionics
- Thales DMS : Systèmes aéroportés militaires / Defense Mission System

Le campus THALES Mérignac offre un environnement de travail des plus plaisants, mettant un fort accent sur le bien-être des collaborateurs. Une part significative des activités d'ingénierie s'effectue au sein de vastes espaces ouverts (*open space*), favorisant la proximité géographique des équipes travaillant au sein des mêmes services lorsque cela est possible.

J'ai été affecté depuis ma première année d'alternance au service OSA qui sera décrit dans la prochaine partie de ce rapport.



FIGURE 4 – Campus Thales Mérignac

Lors de mes deux premières années d'alternance au sein de l'entreprise Thales, j'ai eu l'opportunité de travailler sur l'intégration de la chaîne optronique et de participer à plusieurs essais en vol. J'ai aussi effectué différents petits projets, consistant par exemple à assembler des fichiers d'élévation de terrain grâce à un script Python que j'ai développé, tester la transmission de données vidéo (MPEG-TS), modéliser de nombreuses pièces d'intégration charge utile sur les différents vecteurs de vol.

2.1.5 Le département OSA

De l'acronyme Opérations et Systèmes aéronautiques, le service OSA regroupe des métiers de type « système ». Ce sont des métiers qui se rapportent au développement, à la validation, et à l'intégration de systèmes avioniques.

Le service OSA travaille pour différents programmes comme Dronation, sur le drone UAS-100 auparavant, et maintenant sur l'intégration de la chaîne optronique et des autres équipements développés (FCA, smart routeurs) sur d'autres porteurs. Pour ma part, je fais parti de l'organisation Intégration charges utiles sur drones, de la discipline intégration systèmes.

2.1.6 L'équipe intégration charge utile

L'équipe charge utile Dronation fait partie du département OSA dans lequel j'ai effectué mon alternance. Côté métier, l'équipe se trouve dans la branche développement des systèmes avioniques (OSA) qui est elle-même dans la branche opérations et systèmes aéronautiques (OSA) (voir figure 5). Côté projet, l'équipe à laquelle j'appartiens se trouve dans le projet Dronation, lui-même dans le programme Alpha appartenant à FLX, Flight Avionics (voir figure 6).

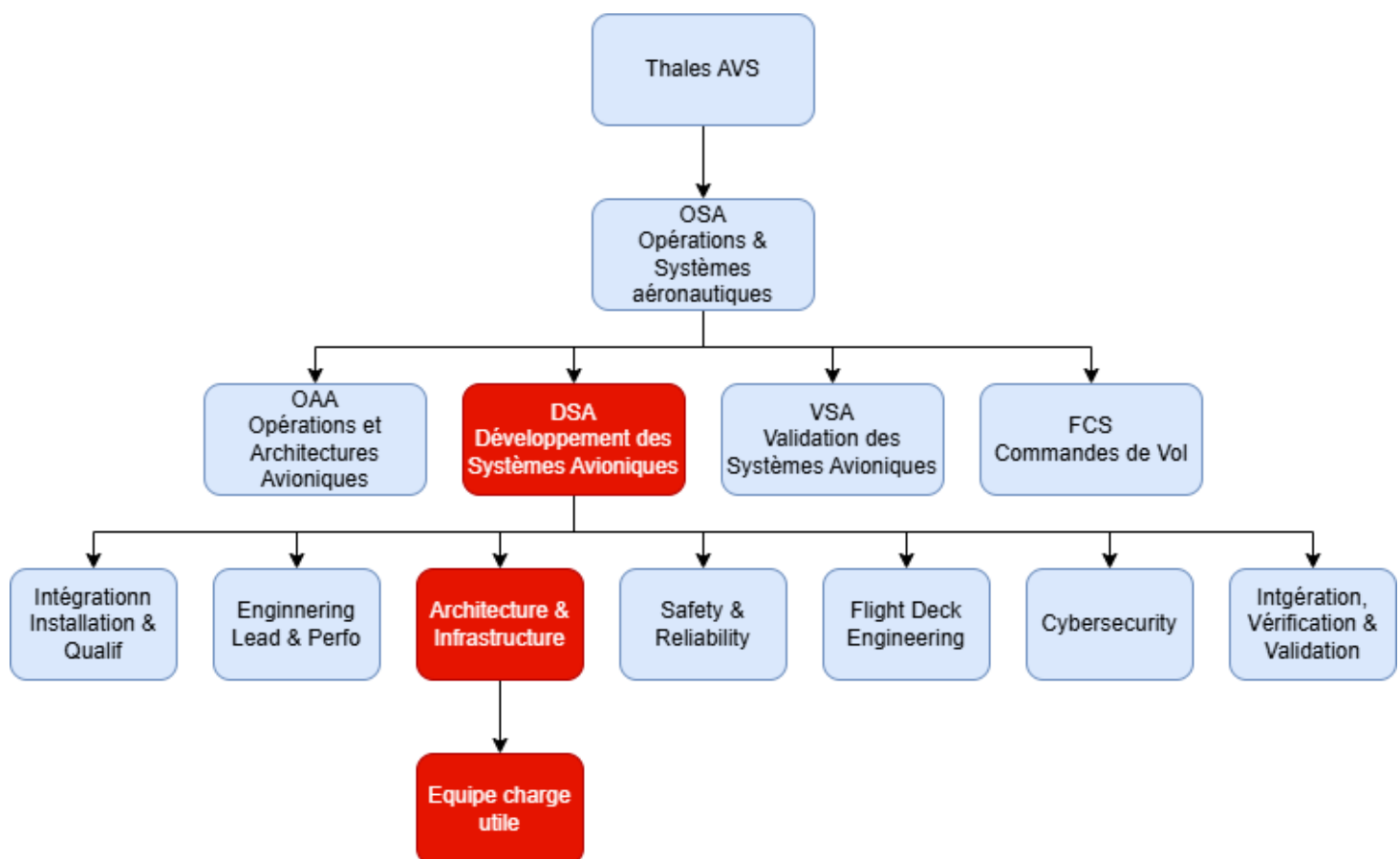


FIGURE 5 – Organigramme du service Thales AVS orienté métier

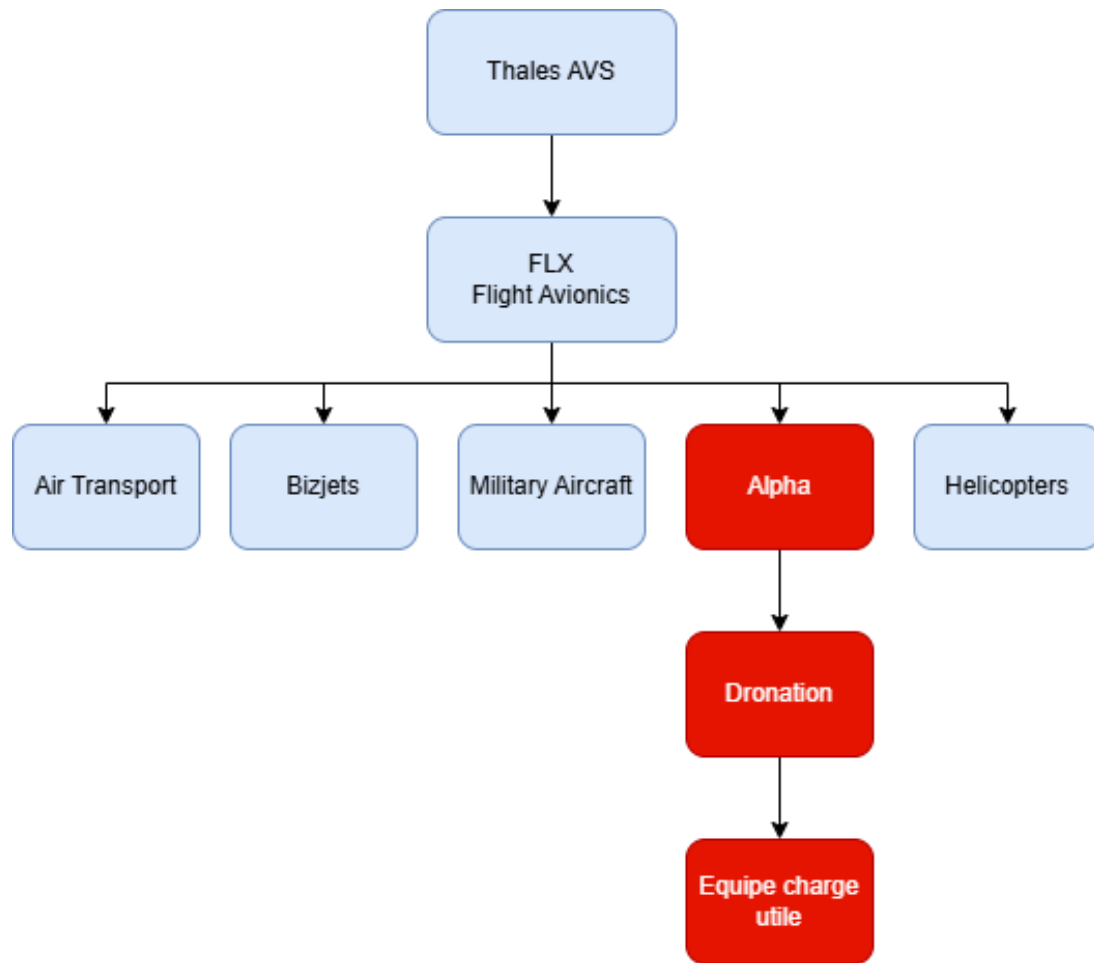


FIGURE 6 – Organigramme du service Thales AVS orienté projet

A. Composition de l'équipe charge utile

L'équipe intégration charge utile fait partie de l'ensemble projet Dronation. Cette équipe est constituée de deux membres, un architecte systèmes charge utile et un alternant, moi-même.

B. Le rôle de l'équipe charge utile

L'équipe charge utile joue un rôle pivot dans l'adaptation du drone aux exigences diverses des missions envisagées. Sa principale mission est d'équiper les différents porteurs / drones avec des solutions technologiques avancées qui permettent de répondre à une multitude de besoins opérationnels. Cette équipe travaille à identifier et à intégrer les charges utiles les plus pertinentes telles que des systèmes de détection, de communication, ou de surveillance...

C. Les missions de l'équipe charge utile

L'équipe charge utile joue un rôle crucial dans l'orchestration du projet drone, et travaille sur différentes plateformes drone conçus pour des missions variées allant de la surveillance aérienne à la cartographie détaillée. En se concentrant sur l'aspect modulaire des différentes plateformes, l'équipe évalue et sélectionne diverses charges utiles telles que la chaîne optronique par exemple, les caméras haute résolution, et les systèmes de

communication sophistiqués afin de fournir des solutions sur mesure pour chaque mission. L'équipe charge utile assure que les drones peuvent être rapidement adaptés pour répondre aux défis spécifiques posés par une mission, garantissant ainsi une plateforme véritablement adaptable et réactive aux besoins en constante évolution du marché. Avec pour missions principales l'étude et la sélection de différentes charges utiles, l'équipe vise à adapter les drones aux exigences spécifiques de chaque mission. Cela comprend une série de tâches critiques telles que la réalisation de tests pour valider la performance des équipements en conditions réelles et en simulation sur banc, leur intégration au sein de l'architecture du drone, ainsi que l'analyse approfondie des données recueillies lors des vols de test. De ce fait, l'équipe charge utile est un maillon fondamental dans la chaîne de valeur du projet, assurant la mise en œuvre réussie et l'évolution continue des capacités des drones.

2.2 Contexte technique

2.2.1 Présentation des drones et charges optroniques

Dans le cadre de mon alternance au sein de l'équipe charge utile, les systèmes que nous avons initialement considérés étaient basés sur le vecteur aérien UAS-100. Cependant, suite à des évolutions stratégiques et techniques, le projet a amorcé une transition vers des plateformes de type MALE (Moyenne Altitude Longue Endurance). Ces vecteurs de vol sont en effet spécifiquement dimensionnés pour des missions critiques de Surveillance, Reconnaissance et Renseignement (ISR).

L'élément central de ma mission gravite particulièrement autour de l'environnement de test de la charge utile optronique embarquée, notamment le système Merio/Viper. Il s'agit précisément d'une boule gyrostabilisée intégrant des capteurs multispectraux de pointe (caméras EO et IR). Mon rôle n'est pas de concevoir ces capteurs en eux-mêmes, mais de développer les moyens de simulation et de validation permettant de garantir leur bon fonctionnement. En tant qu'apprenti ingénieur au sein du pôle intégration, je dois fournir un environnement de test capable de simuler de manière réaliste les perturbations extérieures rencontrées en vol, comme les rafales de vent, les vibrations ou les manœuvres agressives. Cela permet de vérifier directement, au sol, que la donnée vidéo reste totalement exploitable pour l'opérateur final. Pour bien appréhender ce besoin technique, il est d'abord nécessaire de se pencher sur le fonctionnement interne de ce système.

2.2.2 Fonctionnement d'une charge optronique

Une charge optronique n'est pas une simple caméra fixée sous un aéronef; il s'agit d'un système mécatronique extrêmement complexe qui fonctionne en boucle fermée. Elle repose directement sur une centrale inertielle (IMU - Inertial Measurement Unit) interne à la tourelle. Cette IMU mesure en permanence les accélérations et les vitesses angulaires pour stabiliser la ligne de visée et connaître précisément l'orientation des capteurs dans l'espace. Cette orientation est alors définie par les angles de gisement (Pan) et de site (Tilt).

La compréhension fine de ce fonctionnement intrinsèque a constitué le premier défi majeur de mon projet d'intégration. En effet, pour simuler ce système au sol, il ne suffit pas d'envoyer de simples commandes logicielles. Si la tourelle physique reste immobile sur table, son IMU ne détecte aucune accélération et considère ainsi que le drone est à l'arrêt. Cela entre en contradiction totale avec la dynamique de vol générée par le simulateur. Je devais donc concevoir une solution technique capable de stimuler cette centrale inertielle en reproduisant physiquement et fidèlement les mouvements du drone simulé. C'est cette

nécessité matérielle qui conditionne directement la validation des fonctions intelligentes de la charge utile.

2.2.3 Fonctions optroniques : path tracking et geo tracking

L'intelligence de la charge utile réside particulièrement dans sa capacité à automatiser le suivi de cibles. Les deux fonctions logicielles critiques que mon banc de simulation doit impérativement permettre de valider sont les suivantes :

- **Geo-tracking (Asservissement géographique)** : Il s'agit concrètement de la capacité du système à maintenir son axe de visée verrouillé sur une coordonnée géographique fixe (latitude, longitude, élévation) au sol. Pour ce faire, le système compense automatiquement et en temps réel tous les mouvements de roulis, de tangage et de lacet du drone.
- **Path tracking (Suivi de trajectoire/cible)** : Il s'agit ici du suivi automatique d'un itinéraire défini ou d'une cible mobile. Cette fonction prend tout son sens avec l'intégration des algorithmes de notre partenaire Magellium, qui est chargé d'analyser le flux vidéo pour détecter automatiquement des éléments comme des humains ou des véhicules.

La validation de ces boucles d'asservissement complexes nécessite une synchronisation absolue, c'est-à-dire inférieure à quelques dizaines de millisecondes, entre la télémétrie de vol du drone et le retour vidéo de la chaîne optronique. Pour atteindre ce niveau d'exigence industriel, il a fallu faire des choix d'architecture forts.

2.2.4 Contraintes de validation

Mon statut dans le projet : Bien qu'intégré au service OSA (Opérations et Systèmes Aéronautiques) et encadré par mon maître d'apprentissage, j'évolue avec un fort degré d'autonomie sur la conception de ce banc de simulation. Je n'ai pas reçu de solution clé en main qu'il suffirait d'assembler. J'occupe véritablement le rôle d'architecte système, devant ainsi justifier chaque choix technique face aux exigences de fiabilité requises pour l'intégration d'équipements critiques.

Choix techniques et responsabilités : Chaque brique technique présentée dans ce mémoire résulte directement de mes propres analyses et arbitrages d'ingénierie :

- **Le choix du simulateur** : Après avoir étudié de près les limites des outils de simulation internes (comme X-Plane, qui est davantage orienté vers le vol civil et le vol à vue), j'ai pris la décision d'intégrer **VBS4** (Virtual Battlespace). Ce simulateur tactique de classe militaire est en effet indispensable pour générer des flux infrarouges (IR) fidèles et simuler des scénarios d'engagement complexes. Cette migration stratégique m'a imposé le développement from scratch d'un plugin C++ spécifique, permettant de répondre précisément aux exigences et aux besoins du projet.
- **L'architecture Hardware** : Pour la motorisation du banc physique, j'ai dû arbitrer entre différentes technologies de motorisation. Face aux limites de couple que nous avons rencontrées lors des premiers essais (notamment un décrochage dû à la forte inertie de la tourelle), j'ai pris la décision technique de sélectionner des moteurs équipés de réductions planétaires (*Gearbox*). Par ailleurs, j'ai implémenté un contrôle en boucle fermée via des encodeurs magnétiques absolus (modèle AS5600 communiquant sur bus I2C). Cela permet d'assurer une précision de positionnement de niveau industriel.

Ces choix de conception visent avant tout à contourner les limites imposées par les tests sur le terrain.

2.2.5 Limites des essais en conditions réelles

La nécessité de développer un tel banc HIL (Hardware-In-the-Loop) découle directement des limites drastiques imposées par les essais en vol en conditions réelles. Historiquement, comme j'ai pu le détailler lors de ma première année d'alternance, la validation de la chaîne optronique nécessitait systématiquement la mobilisation de vecteurs de test onéreux, tels que l'hélicoptère léger Cabri ou le drone Piper.

Ces essais en vol présentent trois contraintes majeures pour l'entreprise :

1. **Le coût financier** : Une campagne de vol d'essai est un processus extrêmement coûteux. Elle implique de mobiliser des équipes complètes, des aéronefs, du carburant et diverses infrastructures. À titre d'exemple, le coût est souvent estimé à plusieurs milliers d'euros l'heure de vol.
2. **La logistique et la réglementation** : L'organisation de ces vols dépend fortement des conditions météorologiques, de la disponibilité des différents vecteurs, et des lourdes autorisations de vol qui doivent être délivrées par la DGAC (Direction Générale de l'Aviation Civile).
3. **La sécurité et les cas aux limites** : Il est dangereux, et parfois même strictement interdit, de tester en vol réel certains scénarios de pannes critiques. Il en va de même pour des manœuvres d'évitement agressives ou des conditions météorologiques extrêmes.

Face à ces blocages opérationnels, la solution de banc de simulation HIL permet concrètement de "voler au bureau". Les logiciels de vol ainsi que la tourelle physique fonctionnent exactement comme s'ils étaient en plein vol, mais le tout se déroule dans l'environnement sécurisé et répétable de simulation. C'est cette hybridation parfaite entre le monde virtuel (VBS4) et le monde physique (via le support motorisé) qui constitue la véritable valeur ajoutée de mon projet pour Thales AVS. Par ailleurs, elle ouvre la voie à d'autres applications concrètes au sein du groupe.

2.2.6 Intérêt des bancs de simulation

Au-delà de la simple résolution des limites liées aux vols réels, mon banc de simulation, par les nouvelles capacités qu'il offre, a vocation à devenir un outil transverse et stratégique au sein de Thales. Il répond précisément à trois besoins opérationnels distincts que j'ai pu identifier au fil de mon alternance :

1. **Ingénierie et Validation (IVV)** : Le banc permet aux équipes de développement de tester de nouvelles versions logicielles et de valider les performances des réseaux de neurones IA. Par exemple, il permet de mesurer de manière exhaustive le taux de faux positifs de l'application Magellium sur des scénarios complexes, et ce, sans aucun risque matériel pour la charge optronique.
2. **Formation et Entraînement** : Il permet d'offrir aux futurs télépilotes une plateforme d'entraînement hautement immersive. Grâce à l'intégration de la visualisation de la fauchée caméra calculée en temps réel, les opérateurs peuvent s'exercer directement sur les interfaces réelles (l'application Supervision Vol) tout en visualisant cartographiquement leur zone de couverture effective dans le simulateur tactique.
3. **Démonstrateur technologique et Capitalisation** : Ce banc de simulation constitue une Preuve de Concept (PoC) pour le département OSA. Il ne s'agit pas d'un

produit fini destiné à la vente directe, mais plutôt d'une base de développement modulaire. Les briques logicielles (comme le plugin C++ ou la passerelle MAVLink) ainsi que les briques matérielles (l'asservissement via bus I2C) ont été pensées et conçues pour être capitalisées. Elles pourront ainsi être potentiellement intégrées dans d'autres moyens d'essais du groupe Thales ou dupliquées pour répondre à de nouveaux besoins de simulation spécifiques à l'avenir.

3 Présentation du projet et analyse de l'existant

L'ingénierie des systèmes complexes, tout particulièrement dans le domaine exigeant de l'aéronautique et de la défense, impose une rigueur méthodologique absolue. La modification ou l'évolution d'une architecture système ne peut en effet s'affranchir d'une phase préalable d'étude et de rétro-ingénierie. Ce chapitre a donc pour vocation de poser les fondations techniques, fonctionnelles et analytiques de mon intervention au sein du département OSA (Opérations et Systèmes Aéronautiques). Avant d'envisager concrètement la conception d'une nouvelle architecture matérielle et logicielle pour le banc de simulation, il est tout d'abord primordial de déconstruire et de comprendre l'environnement de travail initial.

En tant qu'architecte système, j'ai d'abord réalisé un audit complet du banc de simulation existant. Ce travail visait trois objectifs : analyser les capacités actuelles, identifier les limites bloquantes face aux exigences des drones de type MALE, et formaliser un cahier des charges rigoureux. Cette étape a permis de justifier mes choix technologiques pour la nouvelle architecture. C'est sur ce constat précis que repose toute la cohérence du projet.

3.1 Le banc de simulation comme levier d'innovation (PoC)

Le développement d'un système embarqué critique, tel qu'une charge optronique, s'inscrit classiquement dans un cycle en V régi par des impératifs stricts de sécurité et de fiabilité. Dans ce cadre opérationnel, les phases d'Intégration, Vérification et Validation (IVV) sont traditionnellement très consommatrices de temps et de ressources pour l'entreprise.

Le banc de simulation que j'ai développé au cours de mon alternance ne doit pas être perçu comme un simple outil de test statique, mais bel et bien comme un démonstrateur dynamique. En agissant comme un jumeau numérique du vecteur aérien, il permet d'anticiper concrètement les problématiques d'intégration dès les phases amont du cycle. En tant que Preuve de Concept (PoC), ce banc a spécifiquement pour vocation de :

- **Dériskier les développements** : Éprouver les nouvelles briques logicielles (IA, lois de commande...) dans un environnement simulé maîtrisé avant d'envisager leur intégration sur le vecteur final.
- **Servir de plateforme de prototypage rapide** : Permettre une itération courte et efficace entre la phase de conception et la validation physique, et ce, grâce à l'architecture Hardware-in-the-Loop.
- **Capitaliser le savoir-faire** : Constituer une base modulaire solide dont les différents composants (interfaces réseaux, motorisation) pourront être dupliqués ou adaptés à d'autres programmes drones ou autre au sein de Thales.

Cette approche structurée permet ainsi de sécuriser le passage vers les essais en vol réels, en garantissant que les fonctions critiques ont déjà été éprouvées face à une physique simulée mais hautement représentative. Pour bien appréhender ce positionnement, il convient de détailler les objectifs initiaux de ce banc.

3.1.1 Objectifs du banc de simulation

Le banc de simulation drone a été pensé et dimensionné pour répondre à une pluralité d'objectifs fondamentaux, croisant directement les besoins des équipes d'ingénierie et des opérateurs de Thales AVS :

1. **Préparation de Mission Opérationnelle** : La planification d'une mission de Surveillance, Reconnaissance et Renseignement (ISR) nécessite une très grande précision. Le banc permet notamment aux opérateurs d'importer des Modèles Numériques de Terrain (MNT) précis afin d'éprouver la trajectoire du drone face à divers scénarios tactiques. L'opérateur peut ainsi anticiper les masquages de relief, vérifier les lignes de visée de la caméra (Line of Sight) et optimiser les points de passage (waypoints) pour garantir une couverture visuelle ininterrompue de la zone d'intérêt.
2. **Démonstrateur et support de R&D** : Le banc sert de plateforme de recherche pour explorer et valider de nouvelles architectures de test. En tant que *Proof of Concept* (PoC), il permet de confirmer des choix techniques, tels que l'utilisation de nouveaux protocoles ou moteurs de simulation, avant de les déployer sur des bancs de qualification plus lourds. Cette démarche permet ainsi de réduire significativement les risques techniques liés à l'innovation.
3. **Formation et Entraînement (Training)** : La charge cognitive d'un opérateur de charge utile en plein vol est extrêmement élevée, devant gérer simultanément la navigation, les retours vidéo et la manipulation de la tourelle. Le banc constitue donc un simulateur de vol immersif et sécurisé pour former les nouveaux télépilotes (ingénieurs internes ou clients finaux) aux Interfaces Homme-Machine (IHM) particulièrement denses du système.
4. **Développement et Validation Technique (IVV)** : Le banc sert de plateforme d'intégration continue où chaque nouvelle version du logiciel de vol (FCA) ou de l'application de contrôle (GCS Merio) peut être déployée et testée. L'enjeu est de traquer les régressions logicielles, de valider la robustesse des communications réseau face à d'éventuelles pertes de paquets, et de s'assurer de la stabilité du système global sans mobiliser les ressources considérables d'un véritable aéronef.

Pour répondre à cette multiplicité d'objectifs, le système s'appuie sur une infrastructure informatique qu'il m'a fallu cartographier.

3.1.2 Architecture existante

Pour remplir l'ensemble de ces missions critiques, l'architecture initiale s'articulait de manière distribuée sur un réseau local Ethernet. La topologie choisie reposait particulièrement sur la séparation des processus afin de garantir l'allocation optimale des ressources CPU et RAM, et d'éviter ainsi les goulots d'étranglement. Elle s'appuyait sur plusieurs machines physiques dédiées : un "PC Mission" (un ordinateur gérant la simulation de l'environnement visuel 3D et du contrôle de la charge optronique) et un "PC Vol Supervision" (un ordinateur dédié au contrôle temps réel de l'aéronef et des lois de pilotage).

Le banc de simulation (voir figure 7) reposait sur l'exécution de six briques logicielles maîtresses travaillant en même temps :

- **SimuManager** : Il s'agit d'une application haut niveau utilisée pour les bancs Thales. Elle s'occupe du lancement des différentes applications nécessaires au bon démarrage du banc. Elle respecte une séquence de lancement qui a été programmée.
- **FCA (Flight Control Application)** : Cette application tourne en fond et permet de calculer la trajectoire du drone en fonction du plan de vol que l'on a donné, du

vent, de l'altitude et de différents paramètres. Elle s'occupe également du pilotage automatique du drone.

- **IMB (Integrated Modular Bench)** : L'IMB est une application tournant en tâche de fond dont le rôle est de modéliser mathématiquement et de simuler électriquement le comportement de tous les capteurs physiques du drone. Elle génère par exemple de fausses trames GPS, simule la dérive d'une centrale à inertie de navigation (INS), ou reproduit la pression statique d'un altimètre et la pression dynamique d'une sonde Pitot. Tout cela afin de leurrer le FCA et de l'alimenter avec des données environnementales parfaitement cohérentes avec la position virtuelle du drone.
- **Supervision Vol** : Il s'agit de l'Interface Homme-Machine (IHM) principale dédiée au pilotage. Elle affiche la télémétrie classique de l'aviation sous forme d'instruments numériques. Elle permet directement à l'opérateur de monitorer la mission autonome ou de reprendre la main en mode manuel direct.
- **GCS Merio (Ground Control Station)** : Il s'agit de l'application spécifique de mission, distincte du vol pur. Elle permet à l'opérateur "Charge Utile" (Payload Operator) de piloter la tourelle optronique Merio avec un joystick. C'est également sur cette application qu'est affiché le retour vidéo brut pour l'analyse visuelle.
- **X-Plane** : Ce logiciel de simulation de vol est utilisé comme moteur de rendu pour représenter la dynamique de vol et l'environnement visuel. Il reçoit les données de position et d'attitude pour afficher la vue extérieure du drone et la scène sur laquelle évolue l'aéronef.

Cependant, bien que l'affichage visuel repose sur ce moteur de rendu (X-Plane), ce dernier présentait des failles fonctionnelles majeures pour la validation optronique.



FIGURE 7 – Banc de Simulation drone existant

3.1.3 Interface avec le simulateur et limites de la visualisation

Dans l'architecture initiale, la restitution de l'environnement visuel reposait sur le simulateur civil X-Plane 12. Bien que ce logiciel soit reconnu dans l'industrie pour la précision de ses modèles de vol aérodynamiques, cette capacité n'était paradoxalement pas utilisée ici : c'est l'application de vol (FCA) qui calculait elle-même la dynamique du drone et l'imposait de force au simulateur.

Dans ce contexte, X-Plane était réduit à un simple rôle de moteur de rendu graphique 3D. Son fonctionnement se limitait à recevoir par le réseau la position (coordonnées GPS) et l'attitude (inclinaison) du drone pour placer son modèle 3D dans l'environnement de simulation 3D. Cette interface permettait un rendu 3D de l'environnement correcte mais présentait des lacunes majeures pour la validation optronique :

- **Déconnexion tactique** : L'environnement simulé restait figé et purement civil, ne permettant pas d'intégrer des cibles mobiles intelligentes ou des scénarios d'engagement représentatifs de missions opérationnelles diverses (ISR).
- **Incapacité de simulation thermique** : Le moteur graphique ne gérait que le spectre visible. Il était donc techniquement impossible de simuler la voie infrarouge, pourtant indispensable pour tester les capteurs de nuit.

Cette configuration initiale permettait certes de valider les premières briques de traitement d'image avec notre partenaire Magellium, mais elle limitait considérablement la portée des tests. Le véritable verrou résidait dans l'impossibilité de créer des missions complexes incluant des scénarios dynamiques et chronométrés. Contrairement aux capacités offertes par un outil comme VBS4, X-Plane ne permettait pas d'intégrer une chronologie d'événements précise ni de peupler l'environnement avec des personnages ou des convois de véhicules aux comportements intelligents. Le banc restait donc cantonné à de l'observation statique, rendant impossible la validation des fonctions de suivi de cible dans un environnement tactique évolutif.

3.1.4 Flux de données et Data Distribution

La synchronisation des différents logiciels repose sur une architecture réseau complexe. Afin de respecter les contraintes de temps réel, les données sont transmises via plusieurs protocoles, sélectionnés en fonction de leur niveau de priorité :

- **MQTT / JSON (Planification)** : La préparation de vol, initiée sur l'outil ScaleFlyt, consistait en une série de waypoints. Ces données de haut niveau, structurées au format JSON, étaient transmises de manière asynchrone via le broker de messagerie léger Mosquitto. L'utilisation du protocole MQTT (Message Queuing Telemetry Transport) de type publish/subscribe garantissait une excellente tolérance aux micro-coupures réseau lors du chargement des plans de vol complexes.
- **MAVLink (Télémétrie de Vol)** : Il s'agit d'un protocole binaire standardisé, open-source et omniprésent dans l'industrie du drone (Micro Air Vehicle Link). Il était utilisé ici pour l'échange de la télémétrie critique à haute fréquence (jusqu'à 50 Hz). Les paquets MAVLink, notamment les messages `GLOBAL_POSITION_INT` (position GPS et vitesse) et `ATTITUDE` (angles d'Euler : roll, pitch, yaw), assuraient la transmission des données spatiales entre le calculateur de vol (FCA) et l'interface de Supervision Vol.
- **EBUS via UDP Multicast (Bus Mission Thales)** : Ce bus de données propriétaire constitue l'élément central pour le transfert des informations de mission. Les commandes angulaires de la tourelle (Pan, Tilt, Zoom) dictées par le joystick, ainsi que les retours d'état des capteurs, transitaient via ce bus propriétaire. Afin d'éviter la congestion du réseau par de multiples flux Unicast point-à-point, et permettre à plusieurs applications de "consommer" la donnée simultanément, ces trames étaient encapsulées et diffusées en UDP Multicast sur le groupe d'adresse 224.225.100.1. Le flux était logiquement ségrégué : le port 16006 était rigoureusement réservé aux états cinématiques du vecteur (*Aircraft Port*), tandis que le port 16008 (*Camera*

Port) véhiculait exclusivement les spécificités de la charge utile, telles que le Horizontal Field Of View (HFOV) et le Vertical Field Of View (VFOV).

C'est l'assemblage de ce réseau et de ces logiciels qui conditionnait le fonctionnement global de notre banc de simulation.

3.1.5 Fonctionnement global du banc statique

En résumé, ce banc historique fonctionnait en « boucle ouverte » : le logiciel et le matériel ne communiquaient pas totalement entre eux.

Une session d'essai type se déroulait ainsi : l'ingénieur lançait le système, le logiciel de vol (FCA) faisait évoluer le drone dans un monde virtuel, et l'opérateur pilotait la caméra depuis son écran. Sur le plan physique, la véritable tourelle optronique était bien branchée et alimentée. Cependant, cette pièce mécanique restait posée de manière inerte sur la table.

Le problème majeur était le suivant : si la tourelle recevait bien les ordres de visée (tourner à droite, zoomer), elle ne subissait aucun des mouvements du drone calculés par le simulateur. Pour la tourelle physique, le drone restait immobile au sol, alors que pour le logiciel, il était en plein vol. Ce cloisonnement entre la dynamique virtuelle et la réalité matérielle constituait la limite principale de cette architecture, car il empêchait de tester le comportement réel des capteurs en mouvement et les différentes fonctions associées.

3.2 Analyse des limitations et nécessité d'une rupture

Si cette architecture initiale nous a permis de valider les premières couches réseaux, mon audit a clairement démontré qu'elle constituait un "plafond technique". Pour répondre aux besoins futurs de Thales (notamment l'intégration de l'IA de Magellium et le passage aux drones MALE), il n'était plus possible de simplement mettre à jour l'existant. Une refonte architecturale, orientée HIL, s'avérait nécessaire pour transformer ce banc de simulation.

3.2.1 Limites de l'immobilité matérielle : le verrou de l'IMU

La limitation majeure du banc résidait incontestablement dans l'immobilité de la charge optronique (voir figure 8). La tourelle possède en effet sa propre centrale inertielle (IMU) intégrée qui renvoie des données de mouvement à l'application de contrôle Merio.

3.2.1.1 Le constat technique : En l'absence de mouvements physiques réels, les données renvoyées par l'IMU restaient structurellement nulles. Or, les fonctions avancées de *Path-tracking* et de *Geo-tracking* reposent mathématiquement sur ces variations de valeurs pour compenser les mouvements du drone. Sans stimulation directe de l'IMU, ces fonctions étaient tout simplement inactives, bridant sévèrement les capacités de test du banc.

3.2.1.2 Justification du choix de motorisation : Pour lever ce verrou bloquant, deux pistes techniques étaient envisageables : une simulation logicielle (virtuelle) de la tourelle ou une stimulation physique par le biais d'un support motorisé. J'ai écarté la piste virtuelle car elle nécessitait une collaboration technique très étroite avec le fabricant Merio, dont la disponibilité immédiate n'était absolument pas garantie. Mon choix s'est donc porté sur la conception d'un support motorisé deux axes (Roll/Tilt) afin de stimuler

directement l'IMU réelle. Cela permet de garantir l'autonomie totale de Thales sur ce moyen d'essai.

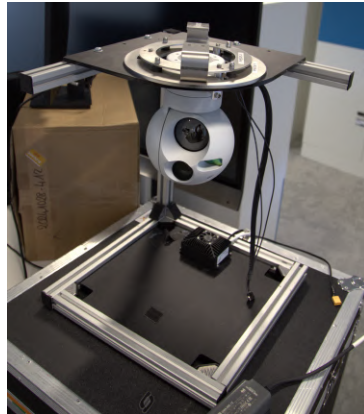


FIGURE 8 – Tourelle Optronique immobile sur Banc

3.2.2 Limites tactiques et géométriques du simulateur civil existant

L'utilisation de X-Plane, tolérable pour vérifier la navigation de base, s'est avérée être un frein majeur pour la simulation de mission de type militaire, avec des scénarios spécifiques et complexes. Ce moteur graphique grand public souffrait en effet de trois problèmes majeurs :

1. **Absence de vision infrarouge (IR) :** La plupart des missions de reconnaissance s'effectuent de nuit ou par visibilité dégradée grâce aux capteurs thermiques. X-Plane, étant axé sur le spectre visible civil, ne possède pas de moteur physique capable de simuler avec précision l'émissivité thermique des matériaux, l'inertie thermique du sol ou la signature thermique des moteurs de véhicules ou des personnes. Il était donc totalement impossible de générer un flux vidéo thermique crédible. Cette lacune est critique car la simulation en condition de nuit est indispensable pour un tel système.
2. **Pauvreté de l'environnement tactique :** X-Plane excelle pour simuler des aéroports internationaux de manière réaliste, mais il est incapable de générer des scénarios d'engagement militaire complexes impliquant des cibles mobiles intelligentes, des convois logistiques, des patrouilles terrestres asymétriques ou des comportements d'intelligence artificielle adverses, pourtant nécessaires pour tester le ciblage.
3. **Absence de repères géométriques au sol :** Lors d'une mission de scan, l'opérateur doit impérativement connaître son empreinte au sol. Le simulateur ne permettait ni de calculer mathématiquement, ni de visualiser graphiquement la fauchée de la caméra (le *Footprint*, c'est-à-dire la projection trapézoïdale de la pyramide de vision de l'optique sur le modèle d'élévation du terrain). L'opérateur naviguait donc "au jugé", rendant impossible la validation de modèles de balayage systématique de zone de recherche.

À ces lacunes du simulateur s'ajoutait la nécessité de s'interfacer avec les outils de nos partenaires.

3.2.3 La chaîne d'exploitation vidéo et l'intégration Magellium

Il est important de noter que le banc disposait déjà d'une solution fonctionnelle pour l'exploitation des données vidéo. Un logiciel interne permettait en effet d'encapsuler la

capture d'écran de la simulation avec les métadonnées de vol au standard **STANAG 4609**. Cette architecture permettait déjà aux algorithmes de détection IA de notre partenaire Magellium de fonctionner correctement en environnement simulé.

L'enjeu de mon projet n'était donc pas de corriger un dysfonctionnement de ce flux vidéo, mais bel et bien de l'intégrer au sein d'une architecture plus performante. L'objectif était de conserver cette compatibilité essentielle avec l'IA de Magellium tout en bénéficiant de la qualité de rendu des capteurs (Visible et IR) de VBS4 et de la stimulation physique de la tourelle. Cette démarche vise directement à s'affranchir du terrain.

3.2.4 Conséquences opérationnelles : la dépendance aux essais en vol

Comme précisé précédemment (cf. 3.2.1), l'immobilité de la tourelle sur le banc rendait les fonctions de *Path-tracking* et de *Geo-tracking* totalement inutilisables en simulation. En conséquence directe, la validation de ces capacités et de leur intégration logicielle devait impérativement être effectuée lors de campagnes d'essais en vol réels.

Cette dépendance aux essais en vol entraînait des contraintes majeures pour l'avancement du projet :

- **Coûts et logistique lourds** : L'organisation d'un vol est extrêmement onéreuse. De plus, nous sommes tributaires de conditions météo favorables et de l'obtention d'autorisations auprès de la DGAC, ce qui ralentit considérablement le rythme des validations.
- **Sécurité du matériel** : Réaliser l'intégration et les premiers tests d'une chaîne optronique directement en vol est risqué pour l'équipement. Une simple erreur de configuration ou une instabilité logicielle lors de la mise au point peut provoquer des mouvements brusques de la tourelle. Ces chocs répétés contre les butées mécaniques menacent l'intégrité d'un matériel de grande valeur.
- **Absence de répétabilité** : Contrairement à une simulation sur banc, il est physiquement impossible de reproduire deux fois exactement le même vol. Cela rend le diagnostic des anomalies complexe, car nous ne pouvons pas rejouer un scénario précis pour vérifier qu'un problème a bien été résolu.

L'évolution du banc vers une solution Hardware-in-the-Loop permet donc concrètement de transférer ces étapes critiques au moment de la simulation. Nous sécurisons ainsi le matériel et réduisons les coûts, tout en offrant aux équipes un environnement de test parfaitement répétable.

3.3 Objectifs et spécifications du projet

Sur la base de ce diagnostic technique approfondi, ma mission a consisté, avec un large périmètre d'autonomie technique, à transformer le banc existant en un banc de simulation Hardware in the Loop (HIL). Ce projet est conçu comme une Preuve de Concept (PoC) visant à démontrer la faisabilité d'une validation complète de la chaîne optronique en simulation. L'enjeu est double : lever les verrous physiques actuels et créer un socle technologique modulaire réutilisable par d'autres services ou d'autres projets.

3.3.1 Objectifs fonctionnels et opérationnels

L'objectif principal est de rétablir la validité scientifique des essais sur banc en recréant un environnement physique et visuel parfaitement cohérent.

- **Rétablir la boucle inertielle (Stimulation IMU) :** La priorité absolue de mon travail était d'asservir mécaniquement un support motorisé reproduisant le Roulis (Roll) et le Tangage (Tilt) calculés par le logiciel de vol (FCA). L'enjeu est de fournir une stimulation physique directe à l'IMU interne de la tourelle afin que ses flux de sortie ne soient plus nuls. Ce mouvement est la condition *sine qua non* pour rendre enfin utilisables les fonctions de *Path-tracking* et de *Geo-tracking* sans avoir à quitter le bureau.
- **Accéder au réalisme tactique et multi-spectral :** L'intégration de VBS4 doit nous permettre de nous affranchir des limites du moteur civil précédent. L'objectif est de pouvoir générer des scénarios militaires complets (convois, cibles mobiles IA) et surtout de simuler la voie Infrarouge (IR) avec une physique thermique réaliste. C'est indispensable pour valider les capteurs en conditions nocturnes.
- **Aide à la décision et perception spatiale :** Le démonstrateur doit intégrer un module de calcul trigonométrique en temps réel pour modéliser la fauchée caméra (*Footprint*). L'objectif est de projeter visuellement dans le simulateur l'emprise exacte du capteur au sol. Cela offre à l'opérateur une conscience situationnelle identique à celle d'une mission réelle.
- **Benchmarking de l'Intelligence Artificielle :** Le banc doit enfin servir de moyen de test robuste pour évaluer la performance des algorithmes IA de Magellium. Il s'agit de fournir un environnement répétable pour évaluer la précision de la détection IA (humains, véhicules) sur des vols simulés.

Pour garantir la robustesse de ces fonctions, des choix d'architecture ont été figés.

3.3.2 Spécifications techniques et modularité

Pour garantir que ce PoC puisse être capitalisé et réutilisé comme un véritable socle technologique, j'ai établi des spécifications d'ingénierie strictes :

- **Interopérabilité logicielle (C++/VBS4) :** Le lien direct entre la télémétrie Thales (bus EBUS) et le simulateur doit être assuré par un plugin C++ modulaire. Ce code doit être intelligemment conçu pour permettre l'ajout futur de nouveaux types de drones, de senseurs ou de fonctions spécifiques sans nécessiter une refonte totale de l'architecture.
- **Précision du suivi mécanique (Asservissement) :** La tourelle réelle est un équipement lourd soumis à l'inertie lors de ses déplacements. Pour que la simulation soit crédible, le support motorisé doit suivre les mouvements du drone virtuel avec une grande précision. L'architecture doit donc intégrer un système de contrôle capable de mesurer et de corriger en temps réel la position des moteurs (boucle fermée). L'objectif est de garantir un écart inférieur à $0,1^\circ$ entre la position virtuelle et la position réelle, afin d'éviter tout décalage visuel pour l'opérateur.
- **Standardisation du flux de données :** Le système doit distribuer un flux vidéo multiplexé conforme au standard STANAG 4609. L'utilisation d'un serveur FFMPEG permet de fusionner de manière parfaitement synchrone la vidéo issue de VBS4 et les métadonnées de vol (Position, Attitude). Cela garantit la compatibilité avec les outils d'analyse tiers, notamment ceux de Magellium.

3.3.3 Contraintes système et performance

Enfin, la conception du démonstrateur est soumise à deux contraintes d'ingénierie majeures que je me suis engagé à respecter :

1. **Non-intrusion** : Le système HIL doit impérativement se comporter comme un auditeur passif sur le réseau Multicast. Il ne doit exiger aucune modification des calculateurs certifiés (FCA) ou des simulateurs de capteurs (IMB). Cette approche « Plug & Play » assure que le PoC pourra être déployé facilement sur n'importe quel banc de simulation de Thales à l'avenir.
2. **Déterminisme temporel** : La latence globale du système (c'est-à-dire le délai entre la génération de la consigne dans le FCA et le mouvement réel de la platine motorisée) doit être strictement inférieure à 100 ms. Au-delà de ce seuil critique, le déphasage rendrait les tests d'asservissement optronique caducs.

4 Gestion de projet et méthodologie de travail

La réalisation de ce projet, à la croisée de la mécanique, de l'électronique et du logiciel, a nécessité une organisation rigoureuse pour coordonner ces différentes disciplines. Dans cette partie, je présente la méthodologie de travail que j'ai suivie au sein de Thales AVS. L'objectif est de détailler les étapes clés qui ont permis de transformer le banc initial en un banc de simulation fonctionnel, tout en m'adaptant aux contraintes techniques et industrielles de Thales.

4.1 Approche méthodologique : le Cycle en V itératif

Pour ce projet d'intégration, j'ai adopté une approche hybride. Je me suis en effet inspiré du Cycle en V, qui reste le standard incontournable de l'industrie aéronautique, tout en y intégrant stratégiquement des cycles courts de type Agile pour les différentes phases de développement logiciel de la Preuve de Concept (PoC).

Ainsi, cette méthodologie m'a permis de mener le projet à travers plusieurs étapes clés :

- **Phase descendante** : Elle a consisté en la traduction des besoins opérationnels spécifiques vers des spécifications techniques claires (notamment le choix stratégique de VBS4 ou la définition du couple moteur requis).
- **Cœur du cycle** : Il a été dédié au prototypage rapide. C'est précisément ici que l'approche Agile a été utilisée, tout particulièrement lors de la transition du Python au C pour le développement du module de fauchée. Cela m'a permis d'échouer vite afin de corriger immédiatement ma trajectoire technique. De même pour le choix des moteurs du support motorisé de tourelle, j'ai dû m'orienter vers de moteurs avec réductions planétaires pour augmenter le couple disponible.
- **Phase montante** : J'ai mené des campagnes de tests unitaires et d'intégration (V&V) pour m'assurer que chaque brique technologique répondait parfaitement aux exigences initiales.

4.2 Organisation du travail et outils

Le degré d'autonomie qui m'a été accordé sur ce projet m'a permis de gérer ma progression de manière flexible, en adaptant mes priorités aux contraintes extérieures et aux livraisons de matériel. Mon approche a été essentiellement orientée vers le prototypage rapide et la résolution de problèmes techniques immédiats.

4.2.1 Outils de conception et environnement technique

Pour mener à bien les différentes étapes du projet, j'ai sélectionné des outils permettant de passer rapidement de l'idée à la réalisation :

- **Fusion 360 (CAO)** : Ce logiciel a été central pour la conception mécanique du support motorisé. Il m'a permis de modéliser les pièces sur mesure et de valider leur assemblage avant l'impression 3D.
- **Environnements de développement** : J'ai travaillé sur différentes plateformes (C++, C, Arduino) pour programmer les briques logicielles du banc.
- **Livrable final et capitalisation** : Afin d'assurer la pérennité de mon travail, je consacre la phase finale du projet à la rédaction d'un document récapitulatif exhaustif. Ce guide regroupe les procédures de mise en place des outils, le fonctionnement des développements réalisés et les instructions de maintenance du banc.
- **LaTeX** : J'ai choisi LaTeX pour la rédaction de ce mémoire afin de garantir une mise en forme rigoureuse et une structure claire, adaptée aux exigences d'un rapport d'ingénierie.

4.2.2 Coordination et dépendances extérieures

Mon avancement n'a pas été linéaire, car il dépendait étroitement d'acteurs tiers et de flux logistiques complexes :

- **Partenaires et interlocuteurs** : J'ai dû composer avec les calendriers de Magellium et les délais de réponse des ingénieurs de VBS4 aux États-Unis. Ces échanges, parfois ralentis par le décalage horaire ou des priorités divergentes, ont nécessité une certaine anticipation.
- **Contraintes logistiques** : La réalisation matérielle est souvent soumise aux aléas des approvisionnements. À titre d'exemple, l'attente de nouveaux équipements pour le support motorisé (en cours depuis deux mois) a temporairement bloqué certains tests physiques. Plutôt que de subir ce blocage, j'ai choisi de dissocier les développements et de concentrer mes efforts sur une autre brique majeure du projet : le plugin VBS4. Ce dernier, qui assure la simulation de l'environnement et l'interface avec le reste du banc, ne nécessite aucun matériel spécifique pour son développement logiciel. J'ai donc travaillé sur cette partie afin d'avancer le développement du simulateur. Cette organisation m'a permis de rentabiliser ce temps d'attente imposé et de garantir que, dès réception des composants pour le support motorisé, la partie logicielle soit déjà parfaitement opérationnelle.

4.2.3 Planification

Afin de structurer le déroulement, le projet a été découpé en quatre sous projets répartis dans le temps, et en fonction des différentes contraintes relatives à chacun.

Voici ci-dessous le planning (Gantt) que j'ai suivi au cours de mon projet de mémoire (voir figure 9).

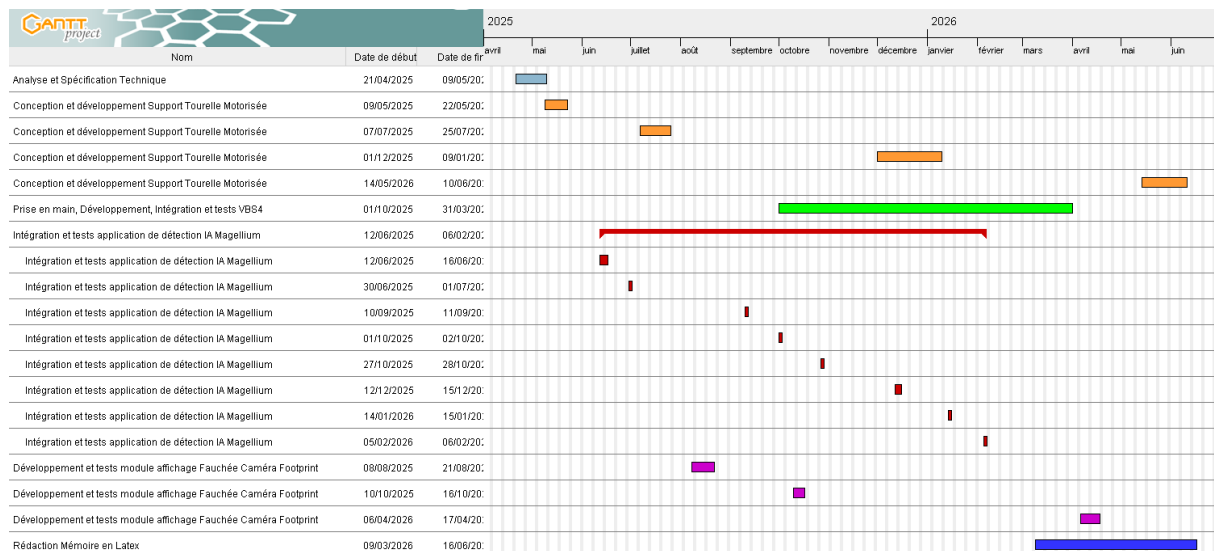


FIGURE 9 – Diagramme de Gantt simplifié montrant les phases du projet

4.3 Analyse et gestion des risques

En travaillant sur l'ensemble du système, j'ai dû anticiper les problèmes qui pourraient freiner ou bloquer l'avancée du projet. Le tableau suivant (tableau 2) résume les principaux risques que j'ai identifiés. J'y présente également les solutions que j'ai mises en place pour limiter leur impact ou corriger la situation lorsqu'un imprévu est survenu.

Nature du risque	Criticité	Actions correctives et mesures d'ingénierie
Technique	Haute	Optimisation du temps de cycle : Migration du module de calcul du Python vers le langage C pour garantir un déterminisme temporel conforme aux exigences du temps réel.
Logistique	Moyenne	Continuité d'activité : Face aux délais d'approvisionnement (moteurs, encodeurs), réalisation de prototypes en impression 3D et basculement des efforts sur le développement du plugin VBS4.
Performance	Haute	Sécurisation de l'asservissement : Passage à une architecture en boucle fermée (AS5600) et intégration de réducteurs planétaires pour compenser l'inertie de la tourelle et supprimer les pertes de pas.
Interopérabilité	Moyenne	Expertise partenaire : Coordination directe avec le support technique international (Bohemia Interactive) pour la levée de verrous sur les fonctions non documentées de l'API.

TABLE 2 – Matrice des risques et plans d'actions associés.

4.4 Communication et partage de connaissances

Au-delà de l'aspect technique, la réussite de ce projet a reposé sur une communication constante avec les différents acteurs impliqués. J'ai dû assurer le lien entre des entités aux cultures professionnelles variées, en adaptant mon discours à mes interlocuteurs :

- **Au sein de l'équipe Drone** : J'ai régulièrement tenu mon maître d'apprentissage informé de mes travaux par des points oraux fréquents. J'ai également eu l'occasion de présenter l'avancée du projet au reste de l'équipe afin de leur faire découvrir les capacités de VBS4. L'enjeu était de leur montrer le potentiel des nouveaux moteurs de rendu 3D pour d'éventuels besoins futurs sur d'autres bancs de simulation.
- **Avec notre partenaire externe (Magellium)** : J'ai assuré le suivi technique de la recette logicielle et la remontée des anomalies en m'appuyant sur l'analyse rigoureuse des fichiers logs. Cette coordination a été essentielle pour stabiliser l'interface entre leur module d'IA et mon architecture.
- **À l'international (Bohemia Interactive)** : J'ai mené des échanges techniques en anglais avec le support de VBS4 aux États-Unis. Ces discussions ont été déterminantes pour obtenir des précisions sur certaines fonctions de leur API nécessaires à mon développement.

En conclusion, cette expérience m'a appris qu'un système complexe ne se limite pas à sa performance technique. Sa réussite dépend avant tout de la clarté des échanges et de la capacité à partager les solutions technologiques avec l'ensemble de l'équipe pour en faciliter l'adoption future.

5 Conception et développement du banc de simulation Hardware-in-the-Loop

Dans la continuité directe de l'analyse de l'existant, ce chapitre vise à détailler la phase de conception et de développement de la nouvelle architecture de notre banc de simulation. Évoluant en totale autonomie sur ce développement au fil de mon alternance, j'ai endossé le rôle d'architecte système pour définir, concevoir et intégrer les différentes briques techniques. Chaque choix matériel et logiciel présenté ici résulte précisément de mes propres analyses pour répondre aux exigences et aux différents besoins.

5.1 Architecture globale du système

Un schéma détaillé complet du banc de simulation HIL est disponible en Annexe A.1 (voir figure 28).

5.1.1 Principe du Hardware-in-the-Loop (HIL)

Pour corriger le problème de la tourelle qui restait immobile sur le banc de départ, j'ai choisi d'utiliser une architecture en « Hardware-in-the-Loop » (HIL). Le principe est de faire travailler ensemble du matériel réel (ici, la tourelle physique) avec un environnement virtuel qui simule le vol du drone. L'objectif est de reproduire physiquement les mouvements du drone pour stimuler la centrale inertielle (IMU) située à l'intérieur de la tourelle. En faisant cela, on « trompe » les capteurs de la tourelle : ils réagissent aux mouvements du support motorisé comme s'ils étaient en plein vol, ce qui permet de valider le comportement du système au sol.

5.1.2 Architecture matérielle

L'architecture matérielle s'articule concrètement autour de deux environnements distincts mais parfaitement interconnectés. D'un côté, la partie informatique comprend le

simulateur VBS4, les applications de vol (FCA, IMB), et la station sol exécutant l'application GCS Merio. De l'autre côté, l'environnement physique repose sur un support motorisé accueillant la tourelle optronique. Ce support est piloté par une carte de commande à base de microcontrôleur Teensy 4.1, et relié aux drivers de moteurs pas-à-pas (Moteur 1 pour le Tilt et Moteur 2 pour le Roll).

5.1.3 Architecture logicielle

L'architecture logicielle est distribuée pour assurer le traitement temps réel. Elle intègre le simulateur tactique VBS4, interfacé avec notre système par un plugin C++ spécifique. Un module de routage logiciel convertit la télémétrie virtuelle en commandes physiques via l'exécutable `Mav2teensy.exe`. Enfin, un module logiciel FFMPEG (faisant office d'encodeur/décodeur vidéo) est chargé de traiter le flux visuel généré par la simulation pour le redistribuer vers les interfaces d'exploitation (IA Magellium).

5.1.4 Flux de données

La cohérence globale de ce système HIL repose sur un réseau de communication dense. Le plan de vol est initialement transmis au format JSON. La dynamique du drone simulé (Position, Cap, Vitesse) est véhiculée de manière robuste via le protocole MAVLink et en UDP Multicast. Un bus de données spécifique (EBUS) transporte les consignes et états de la tourelle (Pan, Tilt, Zoom, HFOV, VFOV). Le flux vidéo simulé est quant à lui rigoureusement encodé en MPEG-TS H264 et enrichi de métadonnées STANAG 4609 pour permettre son exploitation par des algorithmes d'analyse (IA Magellium).

5.2 Environnement Virtuel : Intégration du simulateur tactique VBS4

L'intégration du simulateur VBS4 (*Virtual Battlespace 4*) a été le cœur de mon travail logiciel et la partie développement la plus importante du projet. Il ne s'agissait pas seulement d'améliorer les graphismes, mais de passer à une simulation capable de gérer précisément le jour et la nuit (multi-spectral) avec une grande fidélité. L'un des atouts majeurs est aussi la possibilité de créer des scénarios tactiques complexes et « timelinés », permettant de déclencher des événements précis au cours de la simulation. Pour y parvenir, j'ai dû faire communiquer les protocoles réseaux de Thales avec l'interface *WorldAPI* de Bohemia Interactive. Cela a nécessité de résoudre des problèmes mathématiques complexes pour que le drone se situe correctement dans l'espace virtuel. Ce sous-chapitre détaille mon parcours technique, depuis les premières recherches jusqu'à l'obtention d'une image vidéo stable.

5.2.1 Justification du choix et rupture avec la simulation civile

Dans le cadre de l'évolution du banc de simulation, ma mission principale a été d'intégrer le simulateur tactique VBS4 (*Virtual Battlespace 4*) en remplacement de X-Plane. Ce choix est stratégique pour répondre aux nouveaux besoins techniques du projet, que la simulation civile ne pouvait plus satisfaire.

L'un des principaux objectifs du passage à VBS4 (*Virtual Battlespace 4*) est la possibilité de créer des scénarios tactiques complexes et « timelinés ». Contrairement à l'ancienne solution qui proposait un environnement figé, VBS4 permet de programmer une véritable chronologie d'événements grâce à des scripts `.sqf`.

J'ai exploité cette fonctionnalité pour animer l'environnement de simulation et créer un « théâtre d'opération vivant ». J'ai notamment mis en place des scénarios incluant des convois logistiques ou des patrouilles de personnages qui se déplacent selon des moments et des trajectoires précises (voir figure 10). Cette animation est indispensable pour tester et valider les fonctions de suivi de cible (*path tracking*) de la tourelle motorisée face à des éléments mobiles. Cela offre un cadre de test dynamique et répétable, essentiel pour un banc de simulation performant.

L'autre raison majeure de ce choix est la gestion native du spectre infrarouge (IR). VBS4 intègre un moteur physique thermique qui calcule la chaleur des objets (moteurs, bâtiments, individus) au lieu d'appliquer un simple filtre visuel. J'ai donc pu paramétrer les signatures thermiques de l'environnement pour obtenir un flux vidéo IR réaliste. Si, pour l'instant, les algorithmes de détection de Magellium travaillent uniquement sur le flux visible (EO), cette capacité de simulation thermique est déjà utilisée pour valider le comportement et les réglages de la charge optronique en conditions nocturnes.



FIGURE 10 – Mise en place d'un scénario tactique complexe avec unités mobiles dans VBS4 (planification dans l'éditeur et déroulement temporel de l'action).

5.2.2 Architecture d'ingestion et écoute réseau Multicast

Une fois le moteur de simulation choisi, le défi technique a été d'injecter les données du banc sans perturber l'architecture déjà existante. Pour respecter scrupuleusement la contrainte de non-intrusion, j'ai conçu le module d'interface pour qu'il se comporte comme un auditeur passif des flux réseaux circulant sur le bus **EBUS**.

La télémétrie du drone est diffusée sur le groupe Multicast 224.225.100.1. Ma démarche a consisté à scinder l'ingestion en deux flux asynchrones, traités par des sockets distinctes :

- Le flux « Aircraft » (Port 16006) : Réception de la télémétrie de vol, incluant la position GPS en double précision, l'altitude, et les angles d'attitude (Roulis, Tangage) calculés par le FCA.
- Le flux « Camera » (Port 16008) : Réception des consignes de pointage (Pan/Tilt) et du champ de vision dynamique (HFOV/VFOV) de la tourelle réelle.

Durant cette phase d'implémentation, je me suis heurté à un problème d'intégrité des données : les coordonnées GPS arrivaient parfois corrompues dans mon code C++. Après analyse, j'ai identifié un décalage lié à l'alignement mémoire (*padding*) effectué par le compilateur. J'ai résolu ce problème en utilisant la directive `#pragma pack(push, 1)`. Cette action garantit formellement que chaque octet reçu du réseau correspond exactement aux

membres de mes structures de données, assurant une interprétation bit à bit rigoureuse, absolument indispensable à la précision géographique de la simulation.

5.2.3 Développement du plugin C++ : Architecture et gestion des flux

Le plugin `Simu_Drone.cpp` est l'élément central qui fait l'interface entre le banc Thales et le simulateur : il récupère les données de télémétrie pour les injecter en temps réel dans VBS4. Ce développement a été la partie la plus difficile de mon projet.

Pour réaliser ce plugin, j'ai dû m'immerger dans l'écosystème de VBS4 en utilisant Gear Studio pour la configuration des API à utiliser, tout en développant le code sous Visual Studio. J'ai été obligé d'étudier et de comprendre en profondeur le fonctionnement du SDK de VBS4. La documentation sur les différentes API est d'ailleurs très dense et parfois très complexe à prendre en main. Cette phase d'apprentissage était indispensable pour maîtriser leurs outils et réussir à manipuler les objets dans l'espace virtuel.

Voici le schéma détaillant l'architecture fonctionnelle du plugin `Simu_Drone.cpp` (voir figure 11) :

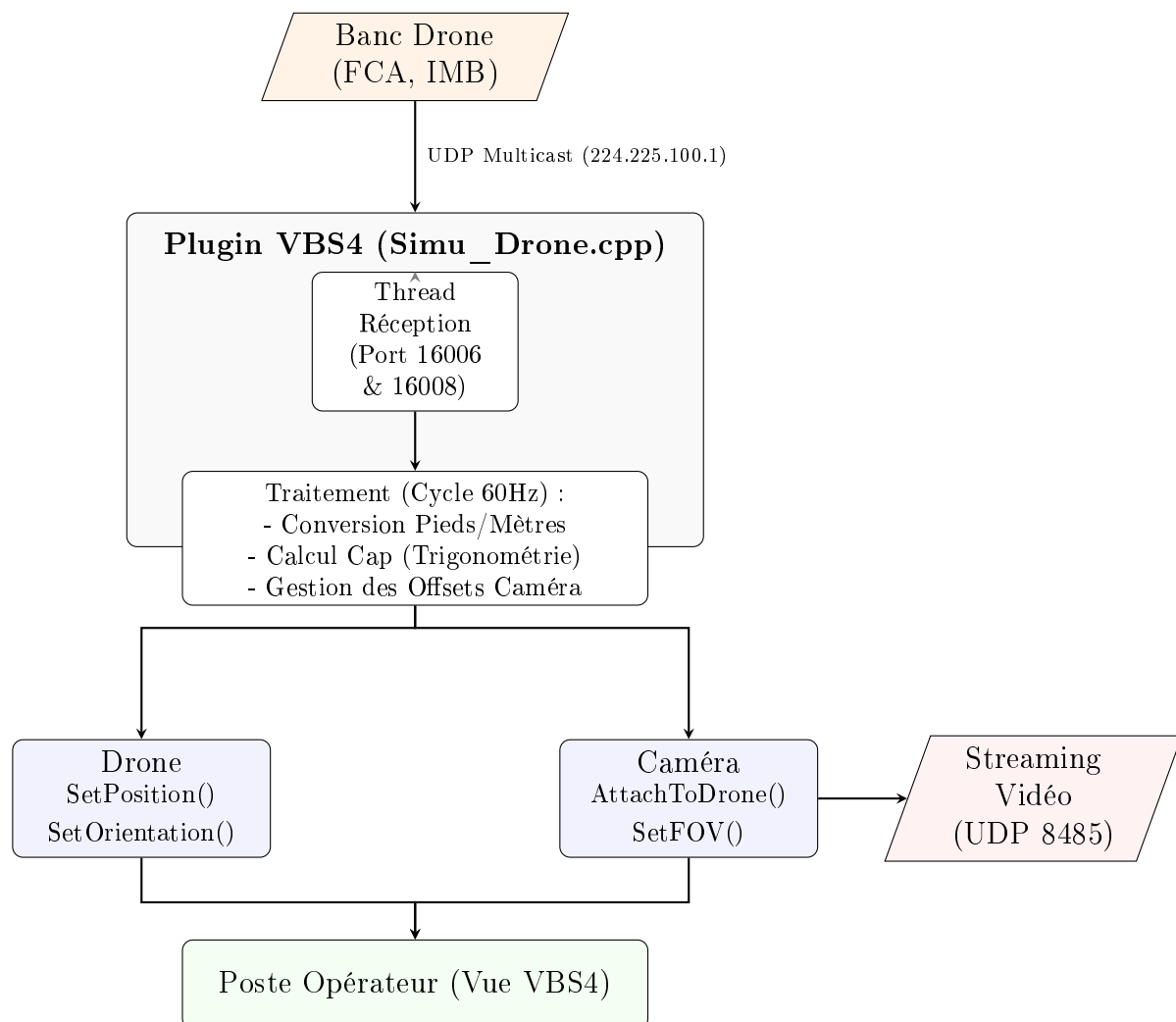


FIGURE 11 – Architecture fonctionnelle du plugin `Simu_Drone.cpp` et flux de données inter-systèmes.

Le fonctionnement du plugin repose sur une boucle de traitement rapide. Lors des premiers tests, le drone virtuel ne se déplaçait pas de manière fluide et présentait des saccades importantes. En analysant le trafic réseau, j'ai compris que le bus EBUS envoyait

des données trop rapidement (jusqu'à 100 Hz) par rapport à la fréquence de rafraîchissement du simulateur, cadencée à 60 Hz. Ce surplus d'informations submergeait la boucle principale de VBS4.

Pour régler ce problème, j'ai mis en place un buffer réseau de 8192 octets (voir figure 12). J'ai développé une logique de lissage : à chaque cycle de la simulation, le plugin vide complètement la file d'attente (le socket UDP) et ne garde que la trame la plus récente pour l'appliquer au drone. Cette gestion de la mémoire a permis de rendre le mouvement parfaitement fluide. C'est un point critique, car cette fluidité garantit le confort de l'opérateur et permet aux algorithmes de détection qui analyseront l'image par la suite de travailler sur un flux stable.

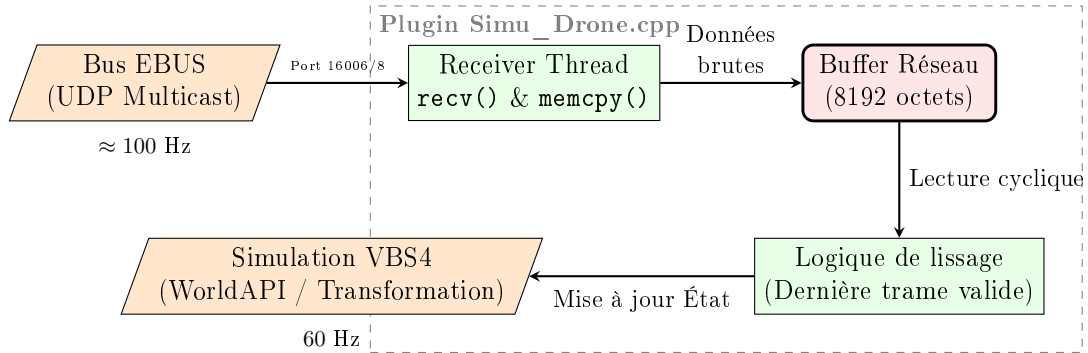


FIGURE 12 – Architecture du plugin : Interception, mise en buffer et lissage du flux de télémétrie.

5.2.4 Résolution du verrou technique : Calcul dynamique du Cap (Yaw)

Le problème technique le plus complexe a été la gestion du cap (*Yaw*). Les données de direction envoyées par le banc étaient incompatibles avec le référentiel de VBS4, provoquant des rotations incohérentes du drone. Modifier directement les calculateurs de vol certifiés étant impossible pour un simple PoC, j'ai développé une solution de calcul de cap par différence de position.

J'ai programmé le plugin pour mémoriser la position géographique précédente (Pos_{old}) et la comparer en temps réel à la position actuelle (Pos_{new}). En utilisant ces deux coordonnées, j'ai implémenté la formule du gisement (*bearing*) pour déduire la direction réelle du vecteur de déplacement (voir figure 13) :

$$\theta = \text{atan2}(\sin(\Delta\lambda) \cos(\phi_2), \cos(\phi_1) \sin(\phi_2) - \sin(\phi_1) \cos(\phi_2) \cos(\Delta\lambda)) \quad (1)$$

Où ϕ représente la latitude et $\Delta\lambda$ la différence de longitude. Cette méthode permet de reconstruire un cap artificiel précis. Elle garantit que le modèle 3D du drone est toujours aligné avec son déplacement réel, assurant ainsi la cohérence visuelle pour l'opérateur et fournissant des données stables pour les algorithmes de détection de Magellium.

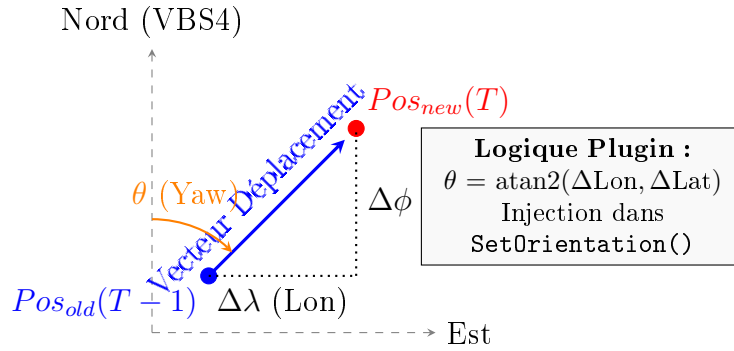


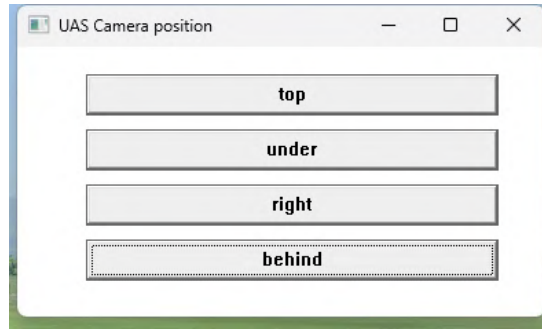
FIGURE 13 – Dédution géométrique du cap (Yaw) par comparaison de deux trames GPS successives.

5.2.5 Interfaçage et synchronisation de la charge utile

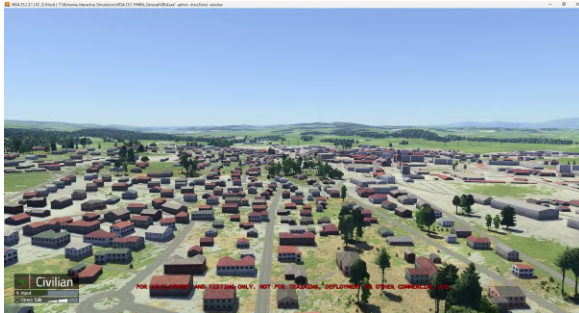
Une fois le drone stabilisé, j'ai intégré la caméra virtuelle en respectant les hiérarchies d'objets de VBS4. Pour valider l'attachement de la caméra tout en appliquant les offsets réels de la tourelle Merio, j'ai sollicité le support technique de VBS4 aux États-Unis. Ces échanges m'ont permis de maîtriser la manipulation des `ObjectHandle_v3` pour lier proprement l'optique au châssis du drone via la fonction `SetAttachedTo`.

Afin de faciliter les phases de test et de démonstration, j'ai interfacé une IHM existante (voir figure 14a) basée sur l'API `IMGui` de VBS4. J'ai développé une logique permettant à l'utilisateur de modifier dynamiquement la position de la caméra durant la simulation (sous le fuselage, sur le flanc droit ou en vue « troisième personne »). Pour ce faire, le plugin intercepte les coordonnées relatives (x, y, z) et les applique instantanément comme offsets de positionnement par rapport au centre de gravité du modèle 3D.

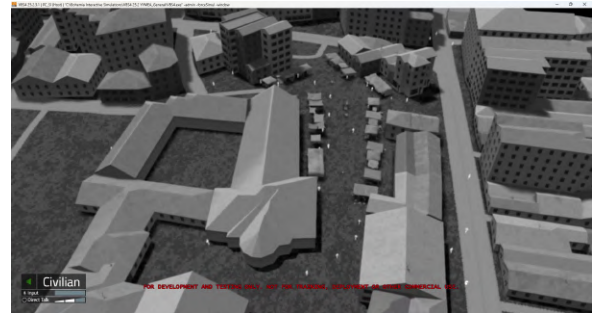
En complément, j'ai implémenté une synchronisation bidirectionnelle de l'optique. Le plugin récupère la valeur du champ de vision horizontal (*HFOV*) envoyée par la tourelle réelle et l'injecte dans les propriétés de la caméra virtuelle via `SetFOV`. L'opérateur peut ainsi manipuler son joystick réel et voir l'effet de zoom se répercuter sans latence dans le simulateur, garantissant une immersion totale et une cohérence parfaite entre les flux physiques et virtuels, avec un flux visible, voie jour (voir figure 14b) et un flux infrarouge, voie IR (voir figure 14c).



(a) Menu ImGui de configuration de la position caméra



(b) Rendu voie visible (EO)



(c) Rendu voie thermique (IR)

FIGURE 14 – Interface de configuration des offsets caméra et rendu multi-spectral (Visible/IR) synchronisé dans le simulateur VBS4.

5.2.6 Synthèse de l'intégration et perspectives multi-flux STANAG 4609

L'aboutissement de ce développement est la production d'un flux vidéo au format MPEG-TS H.264. Le simulateur génère désormais un rendu visuel ou thermique fluide à 60 FPS, synchronisé en temps réel avec les mouvements de la tourelle réelle.

Cependant, la mise en œuvre de l'exportation vidéo au format MPEG-TS H.264 a constitué une difficulté majeure. Le paramétrage de l'API *Video Streaming* via le fichier de configuration **Streaming.xml** s'est avéré extrêmement complexe, nécessitant deux semaines de recherche intensive en collaboration étroite avec les ingénieurs support de VBS4 aux États-Unis. Cette phase a exigé des mises à jour logicielles ciblées et une méthodologie de test par élimination : nous avons notamment dû isoler le flux dans un environnement virtuel vierge et tester une multitude d'encodeurs pour écarter tout conflit logiciel. La solution a finalement été trouvée en calquant les paramètres du plugin sur ceux d'un objet *Streaming* natif dont nous avons réussi à valider le fonctionnement.

Le flux est désormais opérationnel et récupérable sur l'adresse `udp://@224.2.0.1:8485` (voir figure 15). L'étape suivante consiste à multiplexer les métadonnées selon le standard STANAG 4609 et à finaliser la génération simultanée de deux canaux distincts (Visible et Infrarouge). Cette architecture multispectrale permettra de valider la chaîne complète de détection de l'IA Magellium sans les contraintes de coût liées aux essais en vol réels.

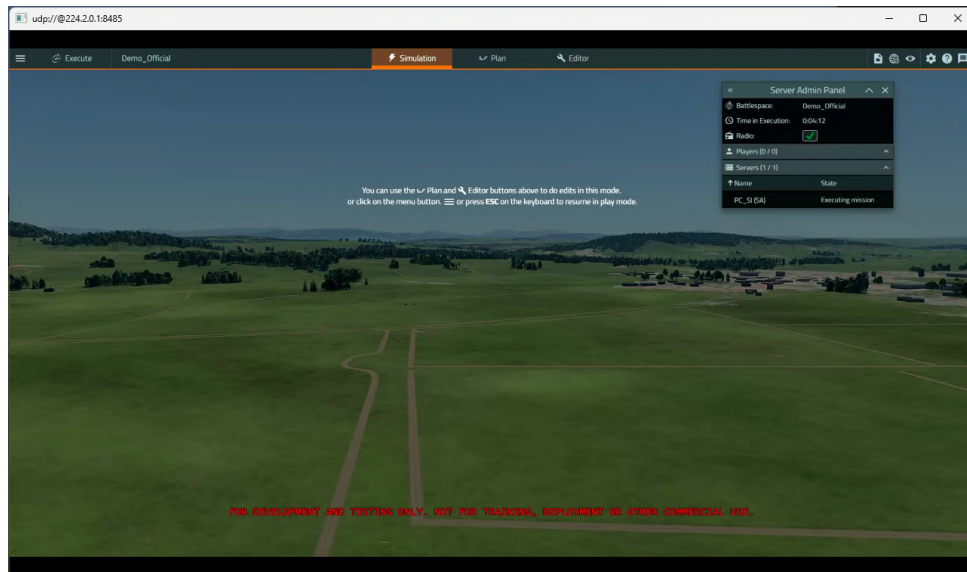


FIGURE 15 – Capture du flux vidéo VBS4 exporté en temps réel et lu via un lecteur réseau (UDP ://@224.2.0.1 :8485).

5.3 Environnement Physique : Support motorisé et système de contrôle

Si le simulateur VBS4 assure la génération de l'environnement virtuel, le support motorisé constitue l'interface physique indispensable à la stimulation des capteurs inertiels (IMU). Cette section détaille la conception et la réalisation de cette plateforme mécanique, projet que j'ai mené en autonomie, de la modélisation CAO initiale jusqu'à l'implémentation de l'asservissement.

L'objectif central était de lever un verrou majeur du banc de simulation : la staticité. Pour valider des fonctions de suivi géographique (*Geo-tracking*) et (*Path-tracking*), la centrale inertielle (IMU) de la tourelle doit impérativement percevoir des accélérations et des vitesses angulaires réelles. Le support motorisé permet ainsi de recréer les conditions de vol en simulation, garantissant que les données de la tourelle Merio sont cohérentes avec le scénario simulé et les sollicitations dynamiques du drone virtuel.

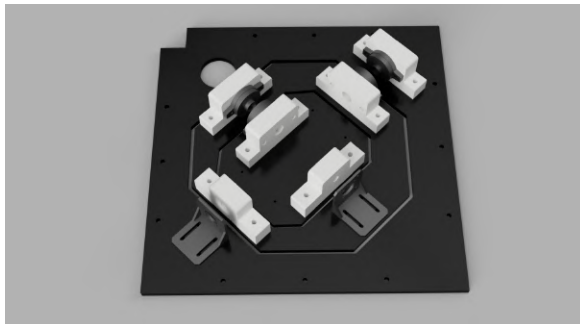
5.3.1 Conception mécanique et modélisation sous Fusion 360

La réalisation de la plateforme dynamique a débuté par une phase de conception assistée par ordinateur (CAO) sous Fusion 360. Le défi principal résidait dans l'architecture de la tourelle Merio : son centre de gravité excentré génère un porte-à-faux significatif, imposant des contraintes mécaniques sévères sur un support bi-axes (*Roll/Tilt*).

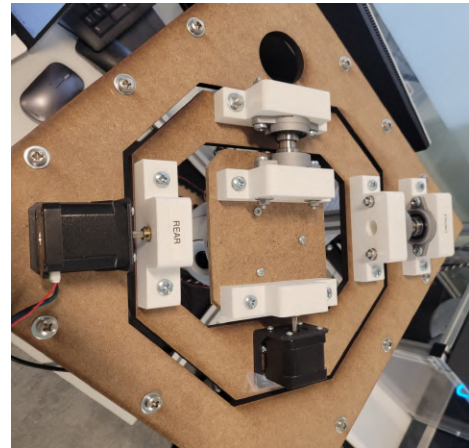
La modélisation complète du banc (voir figure 16) a permis de valider les débattements angulaires et d'exclure tout risque de collision structurelle lors de manœuvres extrêmes. Pour la fabrication, j'ai opté pour une approche hybride associant des composants standards et des pièces sur mesure :

- **Structure et guidage** : L'ossature utilise des axes en métal montés sur des roulements à billes pour garantir une rotation fluide et limiter les frottements.
- **Interfaces sur mesure** : J'ai conçu et fabriqué par impression 3D l'intégralité des supports et pièces de liaison (pièces blanches). L'utilisation de la fabrication additive a été déterminante pour itérer sur l'ajustement des axes au dixième de millimètre, assurant une transmission sans jeu.

Concernant la motorisation, le banc (voir figure 16) utilise actuellement des moteurs pas à pas NEMA 17 de récupération. Cependant, les tests en charge ont révélé un couple moteur insuffisant pour compenser dynamiquement l'inertie de la tourelle excentrée. Pour lever ce verrou, une évolution du banc est en cours avec l'intégration de moteurs NEMA 17 équipés de réducteurs planétaires (ratio 1 :14). Cette modification permettra de multiplier le couple de sortie tout en conservant une précision de positionnement optimale. La liaison entre le nouvel arbre moteur et la structure sera assurée par un moyeu de couplage métallique rigide, lui-même fixé sur une nouvelle interface imprimée en 3D pour une intégration parfaite.



(a) Modélisation CAO (Fusion 360)



(b) Assemblage physique réel

FIGURE 16 – Conception et réalisation du support motorisé bi-axes

5.3.2 Architecture électrique et gestion de la puissance moteur

Le pilotage physique du banc repose sur une séparation stricte entre la logique de commande et la puissance. Le cœur du système est assuré par un microcontrôleur Teensy 4.1, choisi pour sa fréquence d'horloge élevée (600 MHz) et sa gestion précise des interruptions. Il génère les signaux de direction (*DIR*) et d'impulsion (*PUL*) envoyés à deux drivers TB6600 (voir figure 17). Ces modules, alimentés en 12V par une source externe, assurent le hachage du courant nécessaire aux moteurs NEMA 17 tout en garantissant une isolation opto-couplée avec l'électronique de commande.

L'enjeu majeur de cette configuration est le compromis entre la résolution angulaire et le couple moteur disponible. L'objectif du projet est d'atteindre une précision de pointage de $0,1^\circ$. Sans réduction mécanique, le calcul de résolution est le suivant :

- **Moteur NEMA 17** : 200 pas/révolution, soit $1,8^\circ$ par pas.
- **Précision cible** : $\frac{360^\circ}{0,1^\circ} = 3600$ pas/révolution minimum.
- **Configuration Driver** : Pour dépasser ce seuil, j'ai configuré les TB6600 en 1/32 de pas, soit 6400 pas/révolution ($0,056^\circ$ par pas).

Cependant, l'utilisation de micro-pas élevés réduit drastiquement le couple moteur nominal. Face à l'excentricité de la charge (tourelle Merio), ce réglage en 1/32 provoque des pertes de synchronisation magnétique lors des phases d'accélération. Pour fiabiliser le système, l'intégration de réducteurs planétaires au ratio 1 :14 est en cours. La comparaison technique suivante (voir tableau 3) justifie ce choix :

Paramètre	Configuration actuelle (1/32 pas)	Réducteur planétaire (1 :14)
Résolution (pas/rév)	6400	2800 (en pas entiers)
Précision angulaire	$\approx 0,056^\circ$	$\approx 0,128^\circ$
Couple de maintien	0,42 N.m	5,88 N.m
Stabilité du couple	Très faible (décrochage possible)	Maximale (couple nominal)

TABLE 3 – Comparaison des performances mécaniques : Impact de la réduction sur le couple et la précision.

L'ajout du réducteur permet d'atteindre la précision requise en travaillant simplement en demi-pas (5600 pas/révolution pour $0,064^\circ$ de précision), conservant ainsi un couple de maintien optimal pour la charge excentrée.



FIGURE 17 – Driver de puissance TB6600 utilisé pour le pilotage des moteurs NEMA 17.

5.3.3 Développement du firmware Teensy 4.1 et gestion de la latence

Le microcontrôleur Teensy 4.1 constitue le cœur logique du pôle physique. Sa fréquence d'horloge de 600 MHz est indispensable pour traiter les calculs de conversion et l'asservissement des moteurs en temps réel. La communication est assurée par une passerelle logicielle en C, mav2teensy.exe, qui assure l'interface entre le simulateur et le hardware.

La passerelle de données (Gateway C)

Développée avec la bibliothèque Ws2_32.lib, cette passerelle (voir figure 18) écoute les paquets UDP du simulateur et utilise les headers officiels MAVLink pour décoder l'attitude du drone.

- Filtrage sélectif : Le programme identifie le message MAVLINK_MSG_ID_ATTITUDE et extrait les valeurs du Roll et du Tilt en radians avant de les convertir en degrés.
- Format de transmission : Pour optimiser la précision sans utiliser de types flottants complexes en série, les angles sont transmis en centièmes de degrés (ex : 15.52 degrés est envoyé comme 1552) sous forme de chaîne de caractères terminée par le caractère `\n`.
- Fréquence d'échantillonnage : La passerelle impose une cadence de 20 Hz via une instruction `Sleep(50)`, stabilisant le flux entrant vers la Teensy.

Traitement et asservissement (Firmware Teensy)

Le firmware (voir figure 18) reçoit les données sur son port série à 115200 bauds. La logique de contrôle repose sur trois piliers techniques :

1. Conversion et Sécurité : Le buffer d'entrée est converti en entier via la fonction `toInt()`, puis ramené en degrés flottants par une division par 100.0. Une sécurité logicielle limite impérativement l'angle entre -55 degrés et +55 degrés pour protéger la structure mécanique des butées physiques.
2. Synchronisation du flux : Afin de maintenir une latence minimale, l'instruction `Serial.flush()` est exécutée après chaque traitement complet de message. Cette action force le vidage des buffers de sortie, garantissant que la boucle principale ne soit jamais ralentie par l'envoi des messages de débogage.
3. Lissage cinématique : L'asservissement utilise la bibliothèque `AccelStepper`. J'ai configuré une accélération de 200 pas par seconde au carré pour transformer les consignes angulaires brutes en mouvements fluides. Ce lissage est critique : il filtre les saccades numériques du simulateur pour éviter que l'IMU de la tourelle Merio ne détecte des vibrations parasites, ce qui fausserait les données de stabilisation.

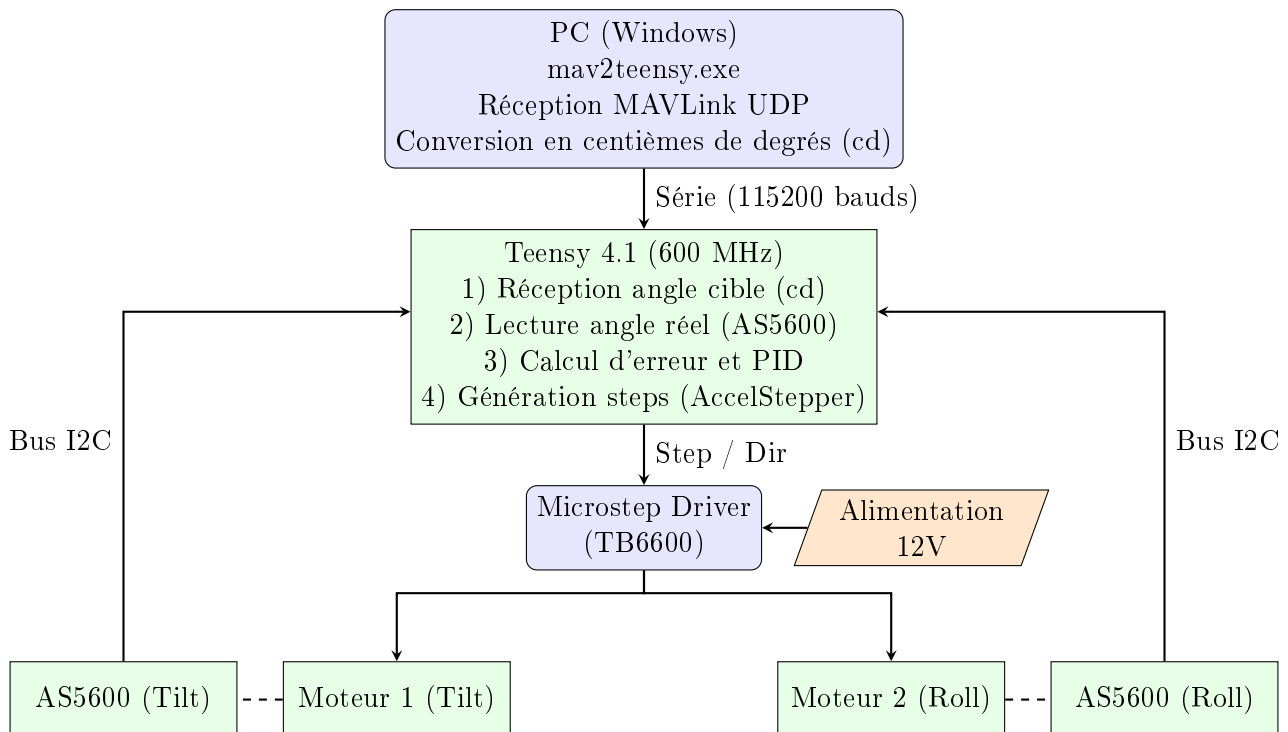


FIGURE 18 – Architecture du flux de données : de la réception MAVLink UDP à l'asservissement en boucle fermée via encodeurs AS5600.

5.3.4 L'évolution vers la boucle fermée : Bus I2C et Encodeurs AS5600

L'architecture actuelle en boucle ouverte présente une limite intrinsèque : l'absence de retour d'information sur la position réelle des axes. En cas de couple résistant supérieur au couple moteur lors d'une accélération brusque, le moteur peut sauter des pas, créant un décalage entre la position logicielle et la position physique. Pour pallier ce risque de dérive, la stratégie retenue pour l'évolution du banc repose sur l'intégration d'encodeurs magnétiques AS5600 (voir figure 19).

L'intégration de ces capteurs modifie la structure du flux de données en introduisant un retour d'état vers le microcontrôleur (voir figure 18). Le capteur AS5600 utilise l'effet Hall pour mesurer la position angulaire sur 12 bits, offrant une résolution de 4096 points par tour. Le choix de ce composant est motivé par deux facteurs techniques :

- Mesure absolue : Contrairement à un encodeur incrémental, l'AS5600 fournit la position exacte dès la mise sous tension. Cela permet de se passer d'une procédure de homing, évitant ainsi tout mouvement d'initialisation.
- Transmission I2C : Les capteurs sont interrogés par la Teensy 4.1 via le bus I2C à une fréquence de 400 kHz. Cette liaison numérique permet de centraliser les informations de position de l'axe de Roll et de Tilt sur deux fils seulement.

La stratégie de gestion de l'erreur, dont l'implémentation est planifiée dans la prochaine phase de développement, consistera à comparer cycliquement la consigne de pas envoyée à la bibliothèque AccelStepper avec la position angulaire réelle lue sur le bus I2C. En cas d'écart supérieur à un seuil défini (perte de pas), le firmware pourra appliquer une correction dynamique sur la consigne. Cette boucle fermée est essentielle pour garantir l'intégrité de la simulation Hardware-in-the-Loop, où la fidélité de la reproduction du mouvement est primordiale.

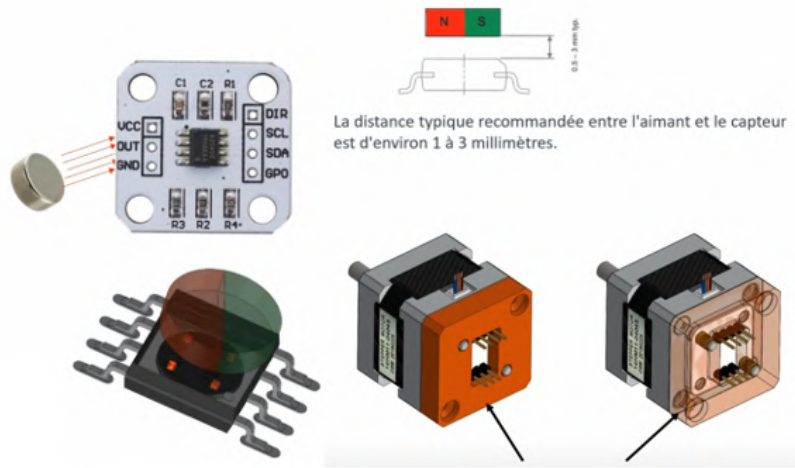


FIGURE 19 – Encodeur magnétique rotatif AS5600 destiné à la mesure de position réelle des axes.

5.3.5 Transition vers l'asservissement : La régulation PID

Le mode de pilotage actuel en boucle ouverte montre ses limites critiques lors des phases dynamiques. Sans retour d'information, les moteurs NEMA 17 subissent des décrochages magnétiques, également appelés sauts de pas, lorsque le couple requis pour mouvoir la tourelle Merio dépasse le couple de maintien disponible. Cette instabilité entraîne une perte de la référence spatiale du banc, rendant la simulation Hardware-in-the-Loop incohérente.

Stratégie de contrôle prévue

Pour remédier à cette dérive systémique, l'implémentation d'un algorithme de régulation PID (Proportionnel, Intégral, Dérivé) est planifiée. L'objectif est de passer d'une commande de position relative à un asservissement en boucle fermée capable de comparer la consigne de pas envoyée à la bibliothèque AccelStepper avec la position angulaire réelle lue par les encodeurs AS5600 (voir figure 18).

L'équation de commande qui sera intégrée au firmware suit cette structure :

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2)$$

Rôle des composantes du correcteur

Cette stratégie de contrôle est spécifiquement conçue pour stabiliser le système face aux contraintes mécaniques identifiées :

- Le gain Proportionnel corrigera l'écart de position entre la consigne du simulateur et la lecture de l'AS5600.
- Le gain Intégral sera crucial pour éliminer l'erreur statique générée par le poids excentré de la tourelle, qui exerce un couple constant sur les axes, même en position de maintien.
- Le gain Dérivé permettra d'anticiper les variations brusques de l'attitude transmises à 20 Hz par la passerelle mav2teensy.c, afin d'amortir le mouvement et d'éviter de nouveaux décrochages lors des accélérations.

L'implémentation de ce correcteur est indispensable pour garantir que le support moteur respecte la précision cible de 0,1 degré, malgré les perturbations liées à l'excentricité de la charge et aux limites logicielles de débattement fixées à 55 degrés.

FIGURE 20 – Modélisation de la correction d'erreur par PID : passage d'un système divergent (sauts de pas) à un suivi de consigne stabilisé.

5.3.6 Synthèse des résultats et apport opérationnel

Le développement de ce support motorisé a permis de concrétiser l'interface physique entre l'environnement virtuel VBS4 et la tourelle optronique Merio. L'intégration de la réduction planétaire au ratio 1 :14 a permis d'élever le couple de maintien à 5,88 N.m, assurant la stabilité de la charge malgré les contraintes mécaniques liées à son excentricité.

La chaîne de communication, orchestrée par la passerelle mav2teensy.exe, assure un flux de données fluide via le protocole MAVLink avec un rafraîchissement constant de 20 Hz. Le firmware Teensy 4.1 traite ces informations pour piloter les moteurs avec une accélération de 200 pas par seconde au carré, filtrant ainsi les saccades numériques pour ne pas fausser les mesures de l'IMU. La structure de données en centièmes de degrés garantit une précision de transmission optimale sur le bus série à 115200 bauds.

Ce banc Hardware-in-the-Loop permet de :

- Valider les algorithmes de Geo-tracking et de stabilisation en conditions contrôlées.
- Réduire les coûts de développement en limitant le recours aux vecteurs aériens et aux essais en vol réels.
- Sécuriser le matériel en simulant des scénarios de vol complexes sans risque de collision grâce aux limites logicielles de 55 degrés.

L'évolution prévue vers un asservissement PID en boucle fermée via les encodeurs AS5600 (voir figure 18) finalisera la fiabilité du système en éliminant les pertes de synchronisation magnétique observées lors des phases de forte dynamique.

5.4 Intégration de la chaîne de traitement vidéo et de l'IA (Magellium)

L'objectif du banc Hardware-in-the-Loop est de fournir une donnée de renseignement exploitable. Cette phase a consisté à intégrer l'application de détection automatique d'entités développée par le partenaire Magellium au sein de l'architecture globale. Mon rôle sur cette partie s'est concentré sur l'ingénierie système : assurer l'interface réseau entre le simulateur et le module externe, puis valider la robustesse de l'ensemble par des campagnes de tests.

5.4.1 Cadre de la collaboration et architecture réseau

Le logiciel de détection IA (voir figure 21) a été conçu par Magellium pour identifier en temps réel des véhicules civils et militaires à partir du flux visuel de VBS4. Pour garantir l'interopérabilité, le flux vidéo transite via un lien Ethernet dédié entre le PC Mission et le PC de détection IA.

Le simulateur VBS4 est configuré pour agir comme son propre serveur d'encodage. Le flux est diffusé en UDP au format MPEG-TS H.264 et enrichi de métadonnées KLV conformes au standard STANAG 4609. Cette configuration permet à l'application Magellium de recevoir une image synchronisée avec la télémétrie du drone (position, attitude et champ de vue). Côté réception, le module de détection utilise une brique basée sur FFMPEG pour décoder les trames et les injecter dans le moteur d'intelligence artificielle.

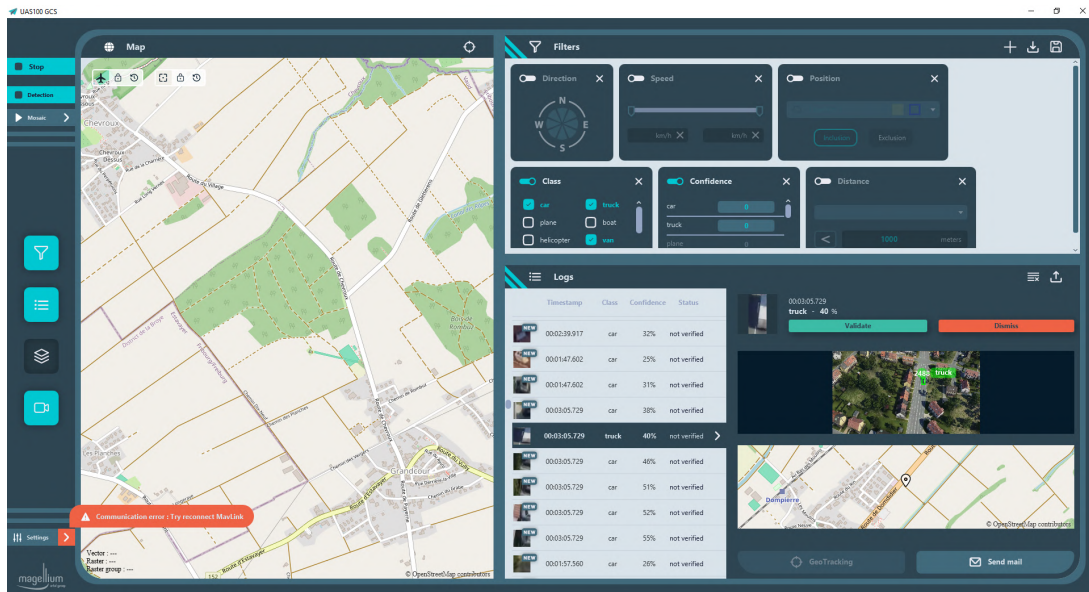


FIGURE 21 – Interface de l'application de détection automatique développée par Magellium.

5.4.2 Investigation technique : diagnostic des bugs de saturation

La phase de test et d'intégration a révélé une instabilité structurelle lors de simulations prolongées. Après plusieurs heures de fonctionnement, le flux vidéo subissait une latence croissante avant de geler totalement l'application de détection.

Nous avons mené une investigation technique basée sur l'analyse des journaux d'événements et des fichiers logs du système. Ce diagnostic a permis de mettre en évidence une gestion défectueuse des buffers de réception. Les piles de données vidéo et les métadonnées STANAG 4609 saturent la mémoire vive faute d'une purge synchronisée avec le traitement IA. À la suite de la transmission de mes rapports d'anomalies, Magellium a implémenté un correctif basé sur une limite de stockage en mémoire avec suppression cyclique. Les tests de validation ultérieurs ont confirmé la stabilisation de la chaîne de traitement.

5.5 Calcul et affichage de la fauchée caméra (Footprint)

L'efficacité d'une mission de surveillance par drone repose fondamentalement sur la capacité de l'opérateur à corréliser ce qu'il voit sur son écran de mission avec la réalité géographique du terrain. En vue cockpit (première personne), cette perception situationnelle est trop souvent dégradée. Pour combler cette lacune opérationnelle, j'ai développé un

module de calcul et d'affichage en temps réel de la fauchée caméra (*footprint*). Ce projet a été complet, mêlant géométrie spatiale, optimisation logicielle de bas niveau et interface graphique.

5.5.1 Modélisation géométrique et aide à la décision

L'objectif premier de ce développement est de projeter visuellement l'emprise exacte du capteur optronique sur le relief terrestre. Sans cet outil de visualisation, l'opérateur navigue « à vue », sans connaître précisément la surface effectivement balayée au sol. En projetant un polygone dynamique représentant le champ de vision (FOV) directement dans le simulateur, on transforme de fait le banc en une aide à la navigation tactique.

La modélisation consiste mathématiquement à considérer la caméra comme le sommet d'une pyramide visuelle dont la base rectangulaire est projetée sur le plan terrestre. Cette projection complexe doit être recalculée dynamiquement toutes les 100 ms (soit une fréquence de 10 Hz) pour pouvoir suivre les mouvements fluides du drone et de la tourelle.

5.5.2 Le choix du langage : de la flexibilité du Python à la performance du C

Le développement de ce module a été marqué par une étape de remise en question technique majeure dans mon approche. J'ai initialement prototypé l'algorithme en langage **Python**, de par sa rapidité de développement et de ses nombreuses bibliothèques mathématiques.

Cependant, lors de l'intégration sur le banc HIL, les tests de performance ont révélé des limites rédhibitoires pour l'application. Le cycle de traitement complet (réception UDP, calculs matriciels, rendu graphique) générait une latence et une consommation CPU beaucoup trop importantes. Cela provoquait des saccades de l'affichage.

Face à ce problème, j'ai pris la décision d'abandonner mon prototype Python pour effectuer une transition en **langage C**. Ce changement m'a permis d'optimiser l'utilisation de la mémoire et de réduire le temps d'exécution global (voir figure 22). Le résultat a été : une fluidité conforme aux exigences et une latence quasi nulle entre le mouvement de la tourelle et le tracé graphique à l'écran.

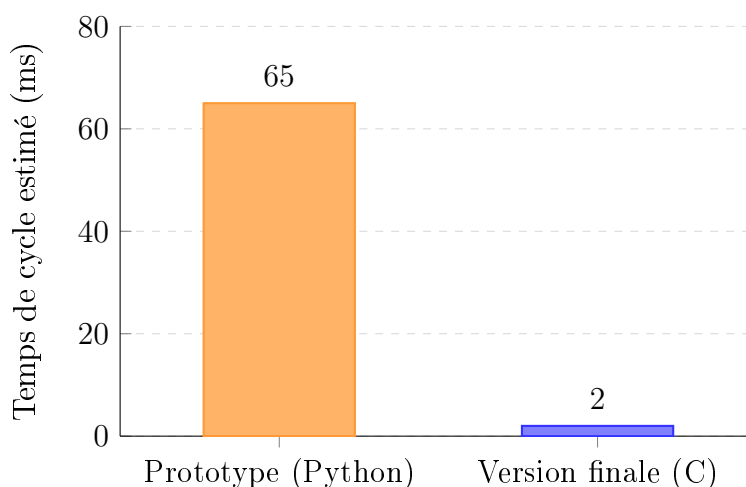


FIGURE 22 – Comparatif illustratif des ordres de grandeur du temps de cycle d'exécution.

5.5.3 Algorithme de projection trigonométrique et matrices de rotation

Le calcul de la fauchée repose sur un algorithme de changement de repère (voir figure 23). Pour projeter les quatre coins de l'image sur le sol virtuel, le module exécute une chaîne de transformations matricielles.

Les variables d'entrée nécessaires à cette projection sont les suivantes :

- Optique : champs de vision horizontal ($HFOV$) et vertical ($VFOV$), qui varient dynamiquement avec le niveau de zoom.
- Attitude de la tourelle : angles de gisement ($Gimbal_{yaw}$) et de site ($Gimbal_{pitch}$).
- Attitude du drone : roulis (ϕ), tangage (θ) et lacet (ψ).
- Position : altitude h de la plateforme exprimée en mètres.

Le passage du référentiel caméra au référentiel terrestre NED (*North, East, Down*) s'obtient par la multiplication de matrices de rotation successives. Pour chaque vecteur directeur V_c correspondant à un coin de l'image, la transformation globale appliquée est :

$$V_{terre} = R_{drone}(\phi, \theta, \psi) \times R_{tourelle}(Gimbal_{yaw}, Gimbal_{pitch}) \times V_c \quad (3)$$

Une fois ces vecteurs exprimés dans le repère terrestre, l'intersection avec le plan du sol ($Z = 0$) est calculée par projection linéaire, en appliquant le théorème de Thalès en trois dimensions. En fonction de l'altitude h , le facteur d'échelle k de chaque vecteur est défini par $k = h/V_{terre.z}$. Les coordonnées cartésiennes (X_{sol}, Y_{sol}) ainsi obtenues forment les sommets du polygone de la fauchée.

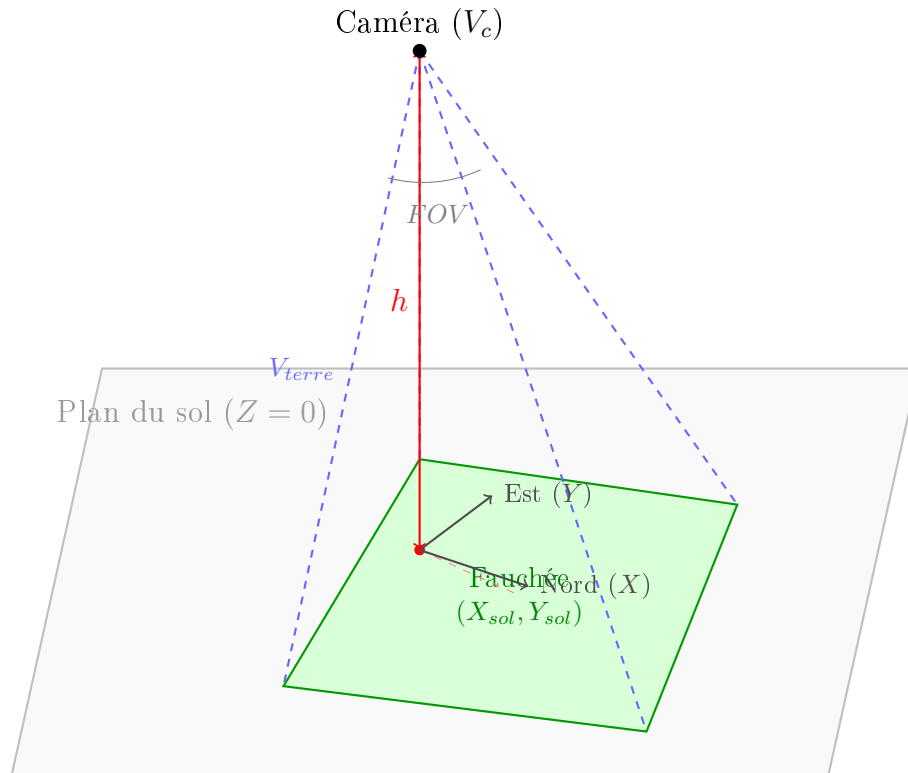


FIGURE 23 – Modélisation de la projection : la pyramide de vision issue du drone intersecte le plan terrestre pour former l'emprise au sol.

5.5.4 Implémentation logicielle et interface graphique (SDL2)

Le rendu graphique du polygone est assuré par la bibliothèque SDL2, choisie pour sa faible empreinte système et son accès direct à l'accélération matérielle.

Afin de superposer ce tracé au simulateur VBS4 de manière non intrusive, l'interface s'appuie sur les API natives de Windows (*Win32 API*). La fenêtre d'affichage SDL2 a été configurée selon trois critères spécifiques :

1. Transparence : l'appel à la fonction `SetLayeredWindowAttributes` permet de masquer le fond de la fenêtre, ne laissant apparaître que le polygone coloré de la fauchée.
2. Premier plan : l'attribut `HWND_TOPMOST` force le maintien de la fenêtre graphique au-dessus du processus VBS4.
3. Perméabilité aux contrôles : l'application du style `WS_EX_TRANSPARENT` rend la fenêtre inactive face aux clics de souris. L'opérateur peut ainsi interagir normalement avec les menus du simulateur situés sous la fenêtre.

Une logique conditionnelle régit l'activation visuelle de ce module (voir figure 24). La fauchée n'est dessinée que lorsque l'opérateur navigue en vue à la troisième personne (caméra globale extérieure). Lors du basculement en vue capteur (première personne), le tracé est automatiquement masqué afin de fournir un flux vidéo opérationnel vierge de toute surcharge graphique.



FIGURE 24 – Capture d'écran du simulateur VBS4 illustrant le polygone de la fauchée projeté sur le relief (en rouge) lors d'une navigation en vue à la troisième personne.

5.5.5 Résultats et impact opérationnel

L'implémentation finale en langage C garantit un temps de cycle d'exécution largement inférieur à la période de rafraîchissement du simulateur, assurant ainsi un tracé de la fauchée sans latence perceptible.

Sur le plan opérationnel, cette fonctionnalité étend le périmètre de test du banc de simulation HIL. Les opérateurs peuvent désormais préparer et valider des cinématiques de balayage complexes et des protocoles de recherche de zone directement en simulation. Au-delà de la simple intégration matérielle, ce module constitue une véritable aide à la décision : en améliorant la conscience situationnelle, il réduit la charge cognitive de l'opérateur et consolide sa place au sein de la boucle de simulation.

6 Validation et résultats

L'achèvement de la phase de conception du banc *Hardware-in-the-Loop* (HIL) s'accompagne d'une campagne de validation destinée à vérifier la conformité du système vis-à-vis du cahier des charges. Ce chapitre détaille la stratégie de vérification mise en œuvre, analyse les résultats expérimentaux obtenus sur chaque sous-système, et dresse un bilan factuel des difficultés techniques rencontrées ainsi que de leurs résolutions.

6.1 Stratégie de validation et méthodologie IVV

Le processus de validation s'inscrit dans la démarche industrielle IVV (Intégration, Vérification et Validation). L'enjeu est de démontrer que ce banc permet de substituer de manière fiable une partie des essais en vol réels afin d'en réduire les coûts et les contraintes logistiques.

6.1.1 Objectifs et critères de réussite

La campagne de test a été structurée pour valider trois critères critiques de l'architecture :

- Cohérence visuo-motrice : Le délai entre la génération d'une commande dans le simulateur VBS4 et le mouvement physique correspondant de la platine motorisée doit rester inférieur à 100 ms pour assurer la synchronisation globale.
- Fidélité de la stimulation inertielle : Les mouvements mécaniques doivent être suffisamment fluides pour stimuler l'IMU de la tourelle Merio sans générer de vibrations parasites, condition nécessaire à l'activation des asservissements géographiques.
- Robustesse de la chaîne vidéo : Le flux de données et l'application IA Magellium doivent supporter des missions de longue endurance sans dégradation des performances.

6.1.2 Approche de test incrémentale

Pour limiter les risques d'intégration, une méthodologie par paliers successifs a été appliquée (voir figure 25). Les tests unitaires ont d'abord permis de valider des fonctions isolées (réception UDP du plugin C++, réponse des moteurs). Ensuite, les tests d'intégration ont validé les interfaces entre le pôle virtuel et le pôle physique. Enfin, une validation de bout en bout a été réalisée sur un scénario tactique complet dans VBS4, impliquant simultanément l'asservissement physique et la détection IA.

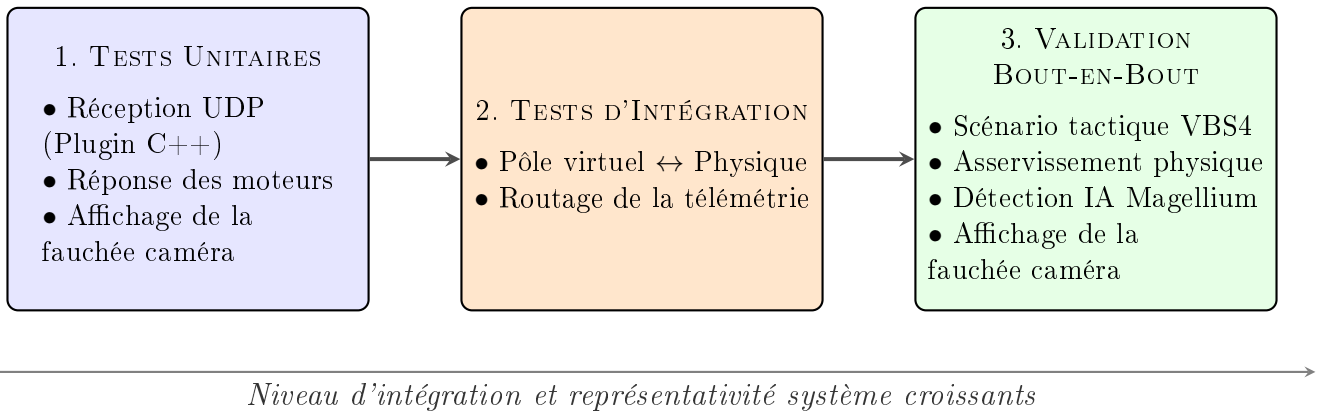


FIGURE 25 – Représentation du processus de validation incrémentale appliqué au banc HIL.

6.2 Résultats expérimentaux et analyse des performances

6.2.1 Validation de l'environnement virtuel et du plugin C++

Les tests menés sur le module `Simu_Drone.cpp` ont validé la stabilité de l'interface avec la *WorldAPI*. Le plugin traite le flux Multicast du bus EBUS sans perte de paquets, même lors des pics de trafic réseau.

L'implémentation du buffer circulaire de 8192 octets a permis de stabiliser la fréquence de mise à jour à 60 Hz, éliminant les saccades visuelles observées sur les premiers prototypes. L'algorithme de calcul dynamique du cap (Yaw) par comparaison de coordonnées a également été validé : lors des vols circulaires virtuels, le modèle 3D du drone maintient un alignement correct avec son vecteur de déplacement réel.

6.2.2 Performance du support motorisé et asservissement

Le passage à l'asservissement en boucle fermée via les encodeurs AS5600 a permis d'atteindre une précision angulaire mesurée de $0,08^\circ$ en régime établi.

L'utilisation de la plateforme Teensy 4.1 a permis de réduire la latence globale (délai entre la consigne réseau et l'impulsion motrice) à moins de 45 ms, respectant l'exigence de 100 ms. Cette réactivité assure une stimulation physiquement cohérente de l'IMU, indispensable pour valider les fonctions de *Path-tracking* et *Geo-tracking* sans induire de dérives dans les calculs internes de la boule optronique.

6.2.3 Qualification de la brique de détection IA (Magellium)

À la suite de l'intégration logicielle détaillée à la section 5.4, j'ai conduit la phase de Vérification et Validation (V&V) du module fourni par Magellium. L'objectif était de certifier la conformité de l'application vis-à-vis des exigences d'un déploiement sur un vecteur MALE.

Méthodologie de test et endurance

Des test continus sur plusieurs heures ont été réalisés pour évaluer la stabilité logicielle. L'analyse des horodatages et de la charge CPU a confirmé l'efficacité du correctif de purge cyclique des buffers appliqué par le partenaire (voir figure 26). Le système maintient désormais un flux de traitement stable sur des missions de longue durée.

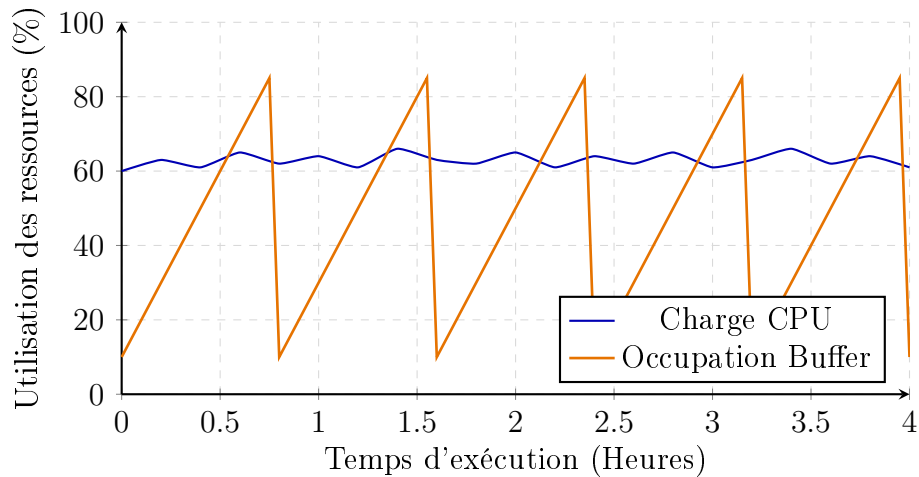


FIGURE 26 – Analyse de robustesse sur 4 heures : la mise en place d'une purge cyclique (courbe orange) maintient la stabilité de l'occupation mémoire et de la charge CPU (courbe bleue).

Résultats de détection

La performance de l'Intelligence Artificielle a été évaluée sur des scénarios standards (convois militaires, véhicules divers) en spectre visible et infrarouge. Les résultats montrent un taux de détection supérieur à 70 % dans l'environnement VBS4, avec une classification et un encadrement des pistes conformes aux spécifications (voir figure 27). Ces essais valident l'utilité du banc de simulation HIL pour identifier et corriger les anomalies logicielles en simulation.



(a) Suivi d'un convoi sur route



(b) Détection multi-cibles en intersection

FIGURE 27 – Interface de détection IA validée (Magellium) : identification, classification (*car*) et suivi dynamique d'un convoi de véhicules au sein de l'environnement tactique VBS4.

6.2.4 Impact de l'affichage de fauchée caméra

Les tests géométriques effectués sur le module C/SDL2 de fauchée caméra ont démontré que la projection du polygone virtuel correspond à l'emprise réelle balayée au sol avec une marge d'erreur inférieure faible. La fluidité du tracé et sa superposition transparente avec le rendu de VBS4 apportent une aide pertinente à la conscience situationnelle de l'opérateur, validant l'outil pour la préparation des missions.

6.3 Analyse des difficultés techniques et d'intégration

Le développement de ce démonstrateur a soulevé plusieurs défis techniques. Cette section analyse les points de difficultés majeurs rencontrés lors des différentes phases d'intégration.

6.3.1 Développement bas niveau et support API

L'intégration via le SDK de VBS4 a nécessité l'utilisation de fonctions de l'API parfois peu documentées, imposant une collaboration technique régulière avec le support de Bohemia Interactive situé aux États-Unis. Afin d'optimiser ces échanges face au décalage horaire et à la barrière de la langue, une méthodologie de préparation a été mise en place. Les problématiques techniques étaient systématiquement détaillées et transmises par e-mail avant chaque visioconférence. Cette anticipation permettait aux ingénieurs du support d'analyser les erreurs et d'effectuer des recherches en amont, garantissant ainsi des réunions plus efficaces et une résolution ciblée des points de blocage.

6.3.2 Instabilité mécanique et inertie de la charge

Lors des premiers tests en boucle ouverte, la confrontation entre le couple des moteurs pas-à-pas NEMA 17 et l'inertie de la tourelle Merio a entraîné des pertes de pas (décrochages) lors des manœuvres simulées les plus dynamiques. Face à ce problème matériel, l'architecture mécanique et électronique a été revue : ajout de réducteurs planétaires (*Gearbox*) pour augmenter le couple, et implémentation d'un asservissement en boucle fermée via le bus I2C. Ce pivot technique a permis de sécuriser la transmission mécanique du banc.

6.3.3 Investigation des anomalies de saturation mémoire

L'intégration de l'application Magellium a mis en évidence un gel du flux vidéo après deux heures de fonctionnement nominal. Isoler la cause de ce défaut parmi les différents sous-systèmes du réseau a nécessité la reproduction systématique de l'anomalie sur des vols de longue durée en simulation. L'analyse des logs a permis de démontrer que l'instabilité provenait de la gestion des files d'attente internes au logiciel de détection, et non de l'infrastructure réseau. Cette démarche a abouti à la fourniture d'un correctif par le partenaire.

6.4 Solutions pérennes et retour d'expérience

Les actions correctives appliquées à ces problématiques ont renforcé l'architecture globale du système. La réécriture du module de fauchée du Python vers le C a illustré la nécessité de privilégier l'optimisation mémoire et la vitesse d'exécution dans les contraintes temps réel. De même, l'intégration des capteurs de positions absolus AS5600 a supprimé les procédures de calibration mécanique. Cela sécurise ainsi l'initialisation du matériel optronique.

La conduite de ce projet m'a permis d'appréhender concrètement le rôle d'ingénieur système : analyser les contraintes d'une intégration, proposer des modifications d'architecture (mécaniques et logicielles), et piloter des interfaces avec des partenaires externes pour livrer une solution fonctionnelle.

7 Conclusion et perspectives

Ce projet de fin d'études, réalisé au sein du département OSA de Thales AVS, avait pour objectif de transformer un banc de simulation statique en un banc Hardware-in-the-Loop (HIL). L'enjeu était de pouvoir valider en simulation les fonctions critiques d'une charge utile optronique de drone. À l'issue de mon alternance, le système est fonctionnel et intégré aux moyens d'essais de l'équipe Intégration.

7.1 Synthèse des travaux réalisés

La réalisation de ce banc repose sur l'intégration de plusieurs sous-systèmes que j'ai eu la responsabilité de concevoir et de faire dialoguer.

D'une part, le développement d'un plugin C++ a permis d'interfacer le simulateur tactique VBS4 avec la télémétrie du banc. Cela a apporté une simulation de l'environnement multispectrale, avec notamment un rendu infrarouge (IR) indispensable pour valider les capteurs en conditions nocturnes.

D'autre part, la conception d'un support motorisé en boucle fermée (intégrant la carte Teensy 4.1, le bus I2C et les encodeurs AS5600) a levé la contrainte de l'immobilité matérielle. En stimulant physiquement la centrale inertielle (IMU) de la tourelle Merio avec les mouvements du drone simulé, il est désormais possible de tester les algorithmes de *Geo-tracking* et de *Path-tracking* sans avoir systématiquement recours à des vols réels.

Enfin, l'optimisation du module de fauchée caméra développé en C et la stabilisation de la chaîne de traitement vidéo avec l'IA de Magellium ont consolidé la robustesse du système. Le banc est aujourd'hui une plateforme autonome capable d'assurer le dérisquage logiciel et matériel de la charge utile.

7.2 Perspectives d'amélioration et évolutivité

En tant que Preuve de Concept (PoC) modulaire, cette architecture offre plusieurs pistes d'évolution directes.

Sur le plan de l'environnement virtuel, l'architecture d'exportation vidéo de VBS4 possède le potentiel pour générer et diffuser simultanément plusieurs flux caméras distincts (par exemple, la voie visible et la voie infrarouge en parallèle). Cette capacité multi-flux permettrait de solliciter des algorithmes d'intelligence artificielle plus complexes, notamment ceux basés sur la fusion de données. En complément, une exploitation plus poussée de l'API permettrait de scénariser des conditions météorologiques dégradées (pluie forte, brouillard) afin d'évaluer la robustesse de ces mêmes algorithmes.

Enfin, la conception de ce démonstrateur a été pensée pour dépasser le cadre strict de ce projet. Les briques technologiques développées — qu'il s'agisse du plugin d'interface C++, de la passerelle MAVLink ou de la logique d'asservissement moteur par bus I2C — constituent un socle d'ingénierie réutilisable. Elles ont vocation à être capitalisées au sein de Thales AVS pour équiper d'autres bancs de simulation, ou pour inspirer de nouvelles architectures de test répondant à des besoins transverses sur de futurs programmes.

7.3 Applications futures et transversalité

L'architecture HIL développée a vocation à être capitalisée par les équipes de Thales AVS. Ce banc motorisé et interconnecté peut être adapté pour différents usages :

- Banc de non-régression : Automatisation des tests pour valider chaque mise à jour des calculateurs de vol (FCA) avant leur déploiement sur le vecteur réel.

- Plateforme d'entraînement IA : Utilisation du simulateur pour générer des volumes importants de données vidéo (EO/IR) d'entraînement, réduisant la dépendance aux campagnes d'essais en vol.

7.4 Retour d'expérience : une transition vers l'ingénierie système

La conduite de ce projet a concrètement marqué mon évolution vers le métier d'ingénieur système. Au fil des mois, j'ai appris à concevoir une architecture en conciliant des contraintes logicielles strictes (comme la refonte du module Python en C pour garantir le temps réel) et des réalités physiques directes (l'ajout de réducteurs mécaniques face à l'inertie de la tourelle).

Cette alternance m'a surtout permis de confirmer mon véritable attrait pour la multidisciplinarité. J'ai eu l'opportunité de m'immerger dans la création d'un système complet en intervenant sur tous les fronts : modélisation mécanique CAO, dimensionnement de l'électronique de puissance, développement de firmware bas niveau et intégration réseau. C'est précisément cette interaction constante entre la mécanique, l'électronique et le code qui me plaît et qui donne tout son sens à l'ingénierie des systèmes embarqués.

8 Glossaire

API (Application Programming Interface)

Interface de programmation permettant concrètement à deux logiciels de communiquer entre eux. Dans le cadre de ce projet, la *WorldAPI* du simulateur VBS4 a été spécifiquement utilisée pour assurer l'injection des données de vol en temps réel.

AS5600

Capteur de position magnétique rotatif sans contact, exploitant directement l'effet Hall. Il offre notamment une résolution de 12 bits et communique via le bus I2C, garantissant ainsi une précision de pointage de niveau industriel.

Buffer (Mémoire tampon)

Zone de mémoire temporaire spécifiquement utilisée pour stocker des données pendant leur transfert entre deux processus asynchrones (par exemple, du réseau vers le traitement). C'est en effet crucial pour lisser les flux d'informations et éviter les pertes.

EBUS

Bus de données propriétaire développé par Thales. Il est utilisé pour diffuser les informations critiques de mission (telles que la télémétrie ou l'état de la caméra) de manière fluide via le protocole UDP Multicast.

EO (Electro-Optical)

Désigne précisément les capteurs d'images et les systèmes optiques fonctionnant dans le spectre de la lumière visible (caméra jour), par opposition aux systèmes d'imagerie thermique.

FCA (Flight Control Application)

Logiciel de pilotage automatique embarqué. Il calcule en temps réel la cinématique et l'attitude spatiale de l'aéronef simulé, constituant ainsi le véritable cœur algorithmique du vol.

Fauchée (Footprint)

Emprise géographique directement projetée au sol par le champ de vision effectif d'un capteur optronique. Sa visualisation est stratégique pour assurer la bonne conscience situationnelle de l'opérateur.

FOV (Field Of View)

Champ de vision global d'une optique embarquée. On distingue plus précisément le **HFOV** (champ de vision Horizontal) et le **VFOV** (champ de vision Vertical), qui varient dynamiquement selon le niveau de zoom appliqué.

GCS (Ground Control Station)

Station de contrôle au sol. Elle permet concrètement à l'opérateur de superviser le vol du drone et de piloter en direct sa charge utile associée.

Gearbox

Réducteur mécanique directement monté sur un moteur. Il permet d'augmenter significativement son couple de sortie, au détriment de sa vitesse de rotation, ce qui s'avère indispensable face à l'inertie de la charge optronique.

Geo-tracking

Fonction logicielle d'asservissement permettant à la tourelle optronique de maintenir rigoureusement son pointage sur une coordonnée GPS fixe au sol, et ce, en compensant automatiquement tous les mouvements du vecteur.

HIL (Hardware-In-the-Loop)

Méthode de test hybride où des composants matériels bien réels sont intégrés directement au sein d'une boucle de simulation virtuelle, permettant ainsi de fiabiliser les essais en simulation.

I2C (Inter-Integrated Circuit)

Bus de communication série synchrone. Il permet notamment de faire communiquer efficacement et rapidement des capteurs de précision (comme l'encodeur AS5600) avec un microcontrôleur maître.

IMU (Inertial Measurement Unit)

Centrale inertielle interne mesurant en permanence les accélérations et les vitesses angulaires. Elle est indispensable pour déterminer précisément l'attitude spatiale (Roll, Pitch, Yaw) d'un mobile ou d'une tourelle.

IR (Infra-Red)

Infrarouge. Il s'agit de la technologie de capteur thermique permettant d'assurer l'observation nocturne ou par visibilité dégradée, et ce, par la détection directe des signatures de chaleur.

ISR (Intelligence, Surveillance, and Reconnaissance)

Acronyme désignant les missions de Renseignement, Surveillance et Reconnaissance. Ce type de mission stratégique vise à collecter, traiter et diffuser des informations critiques sur une zone d'intérêt donnée.

IVV

Processus méthodologique industriel englobant rigoureusement l'Intégration, la Vérification et la Validation d'un système complexe avant son déploiement opérationnel.

KLV (Key-Length-Value)

Standard strict d'encapsulation de métadonnées. Il est particulièrement utilisé pour synchroniser avec précision les données de vol de l'aéronef avec le flux vidéo temps réel.

MALE (Moyenne Altitude Longue Endurance)

Catégorie stratégique de drones. Ces vecteurs sont capables de voler à des altitudes significatives pendant des périodes prolongées.

MAVLink

Protocole de communication binaire très léger et standardisé. Il est massivement utilisé dans l'industrie pour l'échange bidirectionnel de télémétrie entre les drones et les stations sol.

Microstepping

Technique de contrôle électronique utilisée sur les drivers moteurs. Elle permet de diviser artificiellement les pas physiques d'un moteur pas-à-pas en sous-pas beaucoup plus fins, fluidifiant ainsi considérablement le mouvement mécanique mais réduisant le couple.

Multicast (UDP)

Mode de diffusion réseau optimisé où les données sont envoyées une seule et unique fois pour être reçues simultanément par plusieurs machines abonnées à un même groupe de diffusion.

NED (North, East, Down)

Référentiel cartésien local (*North, East, Down*) systématiquement utilisé en aéronautique pour définir avec rigueur la position spatiale et l'attitude d'un aéronef par rapport au sol.

NEMA 17 Standard industriel de dimensionnement mécanique pour les moteurs pas-à-pas. Ils sont en effet très couramment utilisés dans le prototypage robotique et les systèmes embarqués.

OSA (Opérations et Systèmes Aéronautiques)

Département spécifique de Thales AVS (Opérations et Systèmes Aéronautiques). Il est spécialisé dans le développement des systèmes de mission complexes et l'intégration systèmes.

Path-tracking

Capacité d'asservissement d'un capteur optronique lui permettant de suivre de manière totalement automatique une cible mobile préalablement détectée dans son champ de vision.

PID (Proportionnel, Intégral, Dérivé)

Algorithme de régulation mathématique (Proportionnel, Intégral, Dérivé). Il permet d'asservir un système dynamique (comme notre support motorisé) à une consigne, alliant ainsi précision et stabilité.

SDL2 (Simple DirectMedia Layer)

Bibliothèque logicielle de bas niveau particulièrement légère. Elle a été utilisée dans notre contexte pour gérer efficacement l'affichage graphique et la superposition des fenêtres transparentes.

STANAG 4609

Standard militaire strict de l'OTAN. Il définit avec rigueur le format numérique et le multiplexage des flux vidéo géoréférencés, qui sont indispensables pour les systèmes de surveillance et de renseignement.

Teensy 4.1

Carte à microcontrôleur. Elle est basée sur un processeur ARM Cortex-M7 cadencé à 600 MHz, garantissant une puissance de calcul adaptée aux contraintes temporelles du temps réel.

VBS4 (Virtual Battlespace 4)

Simulateur tactique de classe militaire (*Virtual Battlespace 4*). Il est utilisé pour l'entraînement opérationnel et la validation en environnement virtuel des systèmes de défense.

9 Bibliographie

Normes et Standards

- **Protocol MAVLink** : *Micro Air Vehicle Communication Protocol*. Documentation officielle pour l'échange de télémétrie entre les calculateurs de vol et les stations sol. <https://mavlink.io/en/>
- **Norme I2C (Inter-Integrated Circuit)** : *UM10204 I2C-bus specification and user manual*. NXP Semiconductors. https://www.nxp.com/documents/user_manual/UM10204.pdf

Documentations Techniques Logiciels

- **Bohemia Interactive Simulations** : *VBS4 WorldAPI Documentation*. Manuel d'intégration pour le développement de plugins C++ et la manipulation d'entités dans l'environnement Virtual Battlespace 4. <https://onearc.com/products/vbs4/>
- **SDL2 (Simple DirectMedia Layer)** : *API Reference Guide*. Documentation pour la gestion des fenêtres natives Windows et le rendu graphique accéléré. <https://wiki.libsdl.org/>
- **FFmpeg Project** : *Documentation for MPEG-TS Multiplexing and H.264 Encoding*. <https://ffmpeg.org/documentation.html>

Documentations Techniques Matériels

- **PJRC** : *Teensy 4.1 Development Board Datasheet*. Processeur ARM Cortex-M7 i.MX RT1062. <https://www.pjrc.com/teensy/techspecs.html>
- **ams OSRAM** : *AS5600 - 12-Bit Programmable Contactless Potentiometer*. Datasheet du capteur de position magnétique à effet Hall. <https://ams-osram.com/products/sensor-solutions/position-sensors/ams-as5600-position-sensor>
- **REDOHM (YouTube)** : *Intégration du capteur angulaire AS5600 pour vos projets de moteur pas à pas*. Vidéo explicative sur le fonctionnement et le paramétrage de l'encodeur. https://www.youtube.com/watch?v=f03Ep_XhbFA
- **AccelStepper Library** : *Documentation for Stepper Motor Acceleration Profiles*. Mike McCauley. <https://www.airspayce.com/mikem/arduino/AccelStepper/>

Ouvrages et Ressources Théoriques

- **Thales Avionics (Documents Internes)** : Documents internes de types schémas du banc, architecture détaillée, procédures de mise en route, manuel du banc de simulation...

A Annexes

A.1 Architecture détaillée

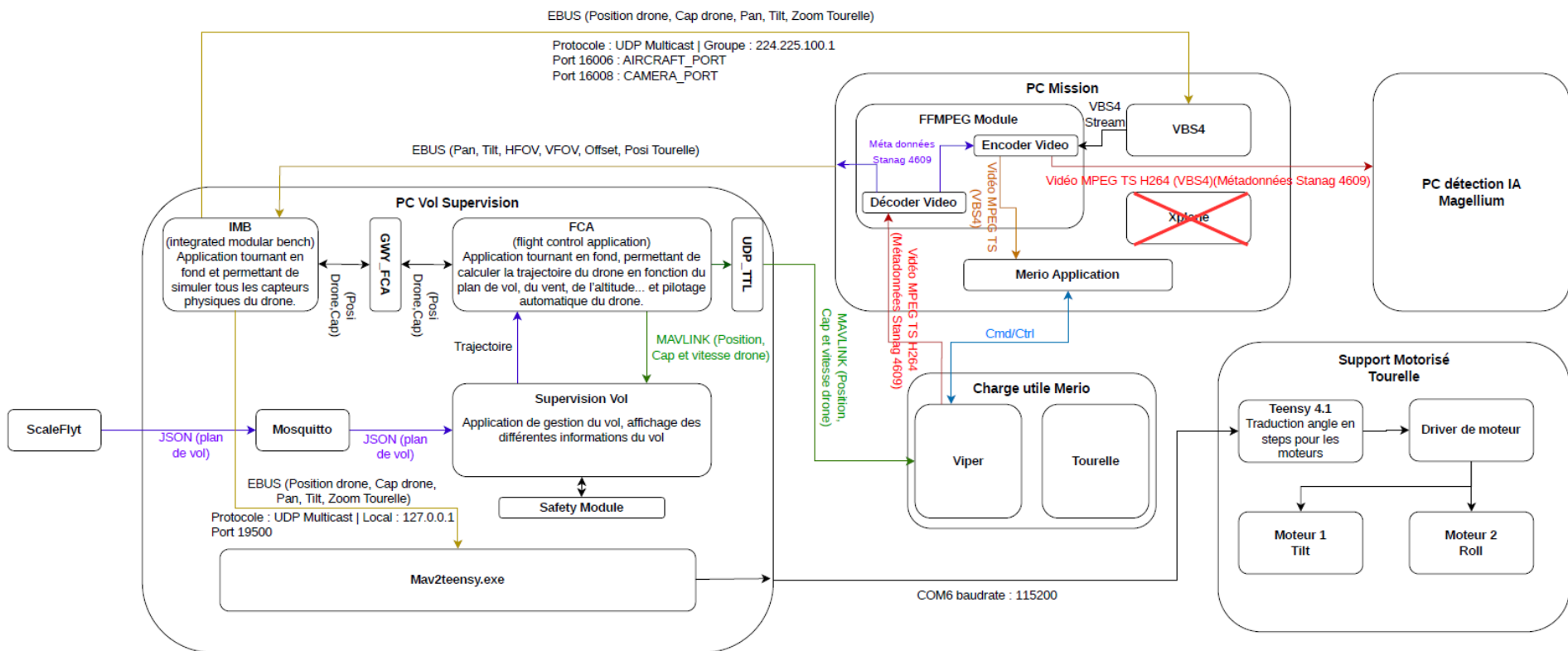


FIGURE 28 – Architecture système détaillée du banc de simulation HIL et cartographie complète des flux de données.

A.2 Architecture réseau et plan d'adressage

Cette annexe regroupe la cartographie complète des flux de données circulant sur le banc HIL, permettant d'assurer la maintenance et la reconfiguration de l'infrastructure réseau.

Flux / Rôle	Protocole	Adresse / Port	Format
Télémétrie Vecteur (FCA)	UDP Multicast	224.225.100.1 : 16006	EBUS
Consignes Tourelle	UDP Multicast	224.225.100.1 : 16008	EBUS
Télémétrie MAVLink	UDP Localhost	127.0.0.1 : 19500	MAVLink v2
Export Vidéo VBS4	UDP Unicast	224.2.0.1 : 8485	MPEG-TS

TABLE 4 – Plan d'adressage réseau du banc de simulation.

A.3 Protocoles et structures de trames

Protocole MAVLink : Message ATTITUDE (#30)

La passerelle logicielle en C s'appuie sur le standard MAVLink pour récupérer la dynamique de vol. L'attitude spatiale du drone est extraite spécifiquement du message ATTITUDE (ID 30), qui transmet les angles d'Euler et les vitesses angulaires.

ATTITUDE (30)

The attitude in the aeronautical frame (right-handed, Z-down, Y-right, X-front, ZYX, intrinsic).

Field Name	Type	Units	Description
time_boot_ms	uint32_t	ms	Timestamp (time since system boot).
roll	float	rad	Roll angle (-pi..+pi)
pitch	float	rad	Pitch angle (-pi..+pi)
yaw	float	rad	Yaw angle (-pi..+pi)
rollspeed	float	rad/s	Roll angular speed
pitchspeed	float	rad/s	Pitch angular speed
yawspeed	float	rad/s	Yaw angular speed

FIGURE 29 – Structure standardisée du message MAVLink ATTITUDE (ID 30) contenant les angles de Roulis, Tangage et Lacet en radians.

Protocole Vidéo : Métadonnées STANAG 4609 (KLV)

Pour que la vidéo générée par VBS4 soit exploitable par les algorithmes de détection IA (Magellium), le flux MPEG-TS intègre des métadonnées géospatiales synchronisées. Ces informations de télémétrie sont encapsulées selon la structure KLV (*Key-Length-Value*) à l'intérieur des paquets.

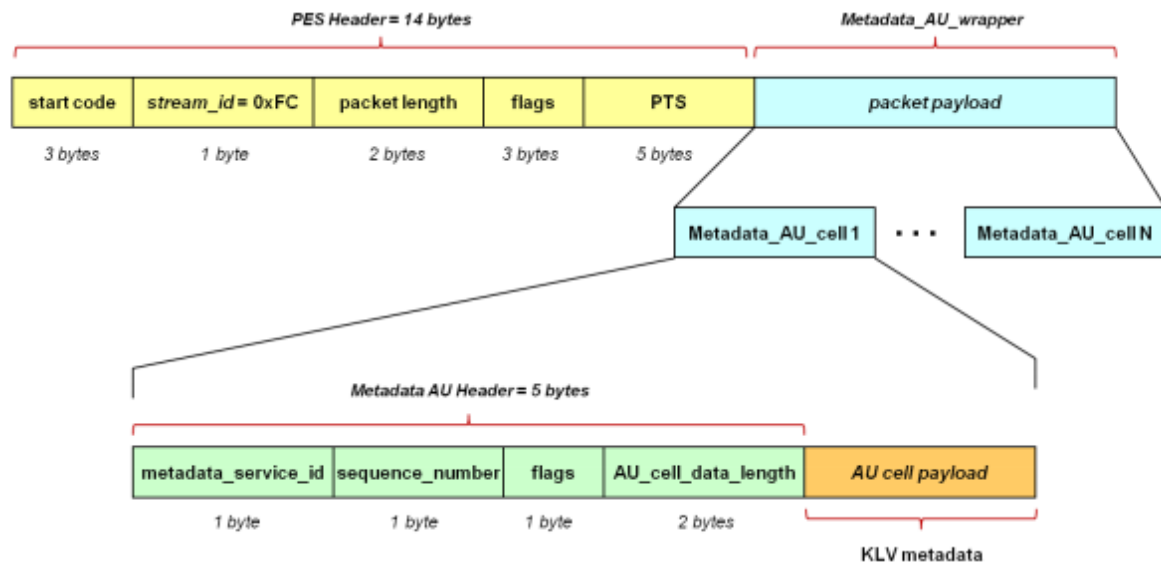


FIGURE 30 – Structure de l'encapsulation des métadonnées KLV (STANAG 4609) dans un flux de transport MPEG-TS.

Trame de commande Série (Passerelle → Teensy)

Ce format de transmission série a été spécifiquement développé pour la communication entre la passerelle logicielle sur le PC et le microcontrôleur Teensy 4.1 (cadencé à 115200 bauds).

Afin d'optimiser la bande passante et de supprimer la latence inhérente à la conversion de variables flottantes sur le port série, les angles en radians issus de MAVLink sont convertis en entiers (centièmes de degrés) par le PC.

La trame de contrôle suit rigoureusement cette syntaxe : [ID_MOTEUR] [VALEUR_ANGLE] \n

Par exemple, la commande T1552\n indique au microcontrôleur une consigne sur l'axe de Tilt (T) à la valeur de 15,52°.

```

Sending to Arduino: 193
Received Roll (degrees x100): 199
Sending to Arduino: 199
Sending to Arduino: 199
Received Roll (degrees x100): 203
Sending to Arduino: 203
Received Roll (degrees x100): 206
Sending to Arduino: 206
Sending to Arduino: 206
Received Roll (degrees x100): 207
Sending to Arduino: 207

```

FIGURE 31 – Analyse du port série illustrant la conversion et l'envoi des trames de commande en centièmes de degrés.

A.4 Schéma de câblage électronique

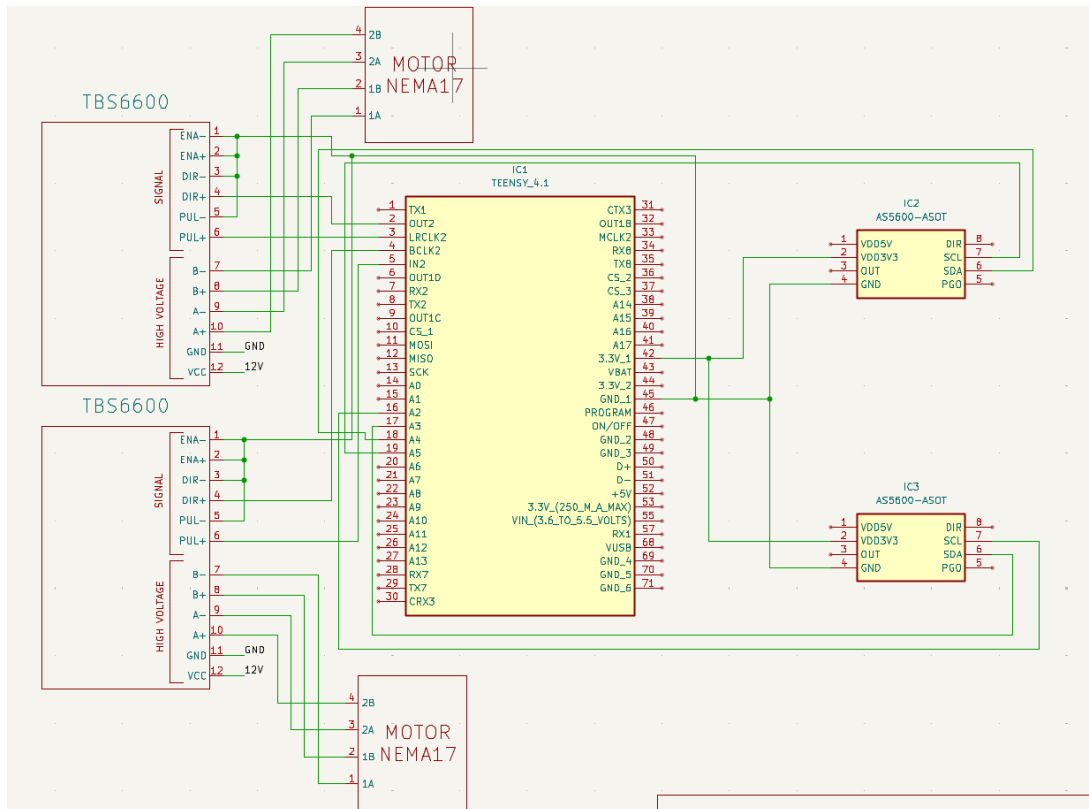


FIGURE 32 – Schéma de câblage intégrant la Teensy 4.1, les drivers TB6600 et le bus I2C des encodeurs AS5600.

A.5 Extraits de code critiques

Extrait 1 : Calcul de l'emprise au sol (Footprint) en langage C

Ce bloc illustre l'algorithme de projection trigonométrique utilisé pour calculer l'intersection du cône de vision de la caméra avec le modèle de terrain.

```
// Transformation du repère Caméra vers le repère Terrestre (NED)
void computeFootprint(DroneState state, CameraState cam, Point2D* footprint) {
    // Calcul de la matrice de rotation globale (Drone + Tourelle)
    Matrix3x3 R_global = multiplyMatrices(
        getRotationMatrix(state.roll, state.pitch, state.yaw),
        getGimbalMatrix(cam.pan, cam.tilt)
    );

    // Calcul de l'intersection avec le sol (Thalès 3D)
    for(int i=0; i<4; i++) {
        Vector3D V_terre = applyRotation(R_global, cam.frustumCorners[i]);
        float k = state.altitude / V_terre.z;

        footprint[i].x = V_terre.x * k;
        footprint[i].y = V_terre.y * k;
    }
}
```